

CIS 023

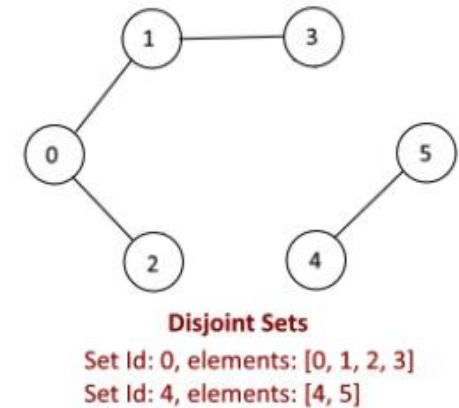
a preview of what data structures & algorithms is about

union-find

our first algorithm design test case

Here's the problem we want to solve:

- We have a set of elements that can potentially be connected to each other. We want to be able to ask whether any two elements in this set *actually are* connected.
 - If you'd like, you can call the elements “vertices” or “nodes”, and the connections “edges” or “links.”
- If it is possible to travel from one element to another by following connections, then we say that there is a *path* between those elements
- The textbook gives elements numbers for names, starting with 0 and ending at $n - 1$, where n is the number of elements in our set.



2 and 3 are connected, because there is a path between them.

4 and 0 are *not* connected, because there is no path between them.

Why do we want to solve this sort of problem?

- There's a huge array of problems where we might want to get a quick response to the question "are these two things connected?"
- We might want to know if two people are in the same social circle, i.e. can we get from one to the other through friend-of-friend relations
- We might want to know if two people are related in a genealogical sense – i.e. if you go far back enough, are they in the same family tree?

Why do we want to solve this sort of problem?

- Bank fraud problem:
 1. Say that we're analyzing a batch of 100,000 recently opened bank accounts.
 2. These accounts may be "related" in various ways
 1. Two accounts A and B were opened from the same laptop
 2. B and C used the same phone number to verify
 3. C and D logged in from the same IP address
 4. D and E share a mailing address...

Bank fraud example

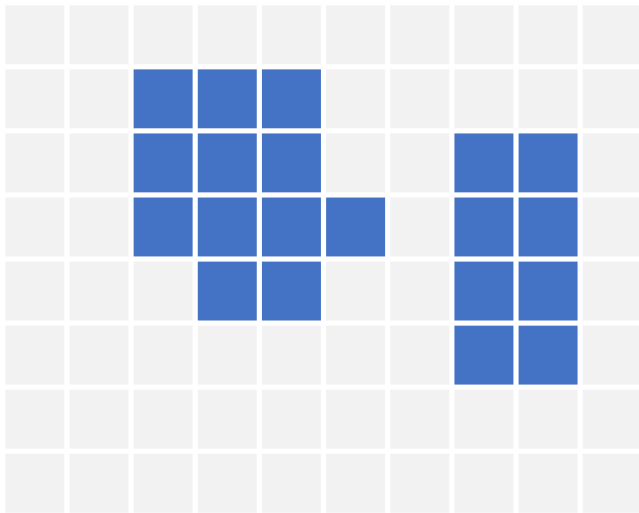
These accounts may be "related" in various ways

1. Two accounts A and B were opened from the same laptop
2. B and C used the same phone number to verify
3. C and D logged in from the same IP address
4. D and E share a mailing address...

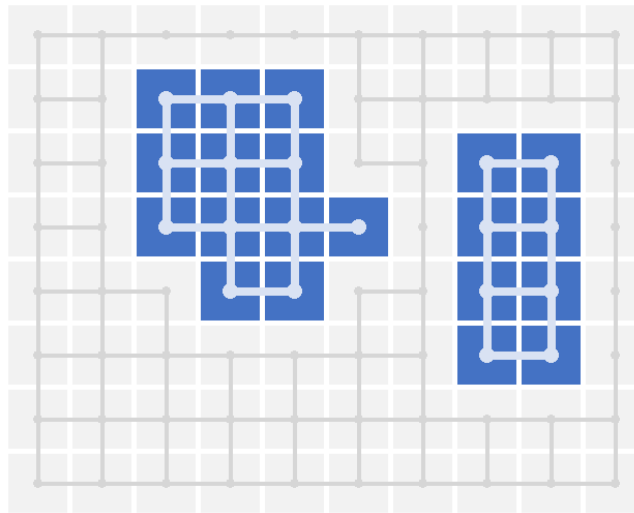
If we can quickly answer the question "is there a way to follow connections to A and reach E", we can hypothesize that A and E were opened by the same person. That's easy in the example above, where we only talk about A, B, C, D, and E, but if we were digging through 100,000 recently opened accounts, it might prove very hard to find these connected ones.

an application: the paint bucket tool

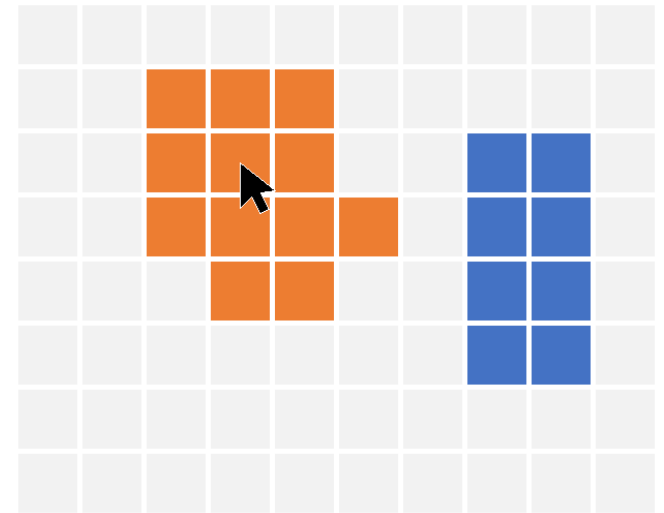
The flood fill (“paint bucket”) tool in an image editor is union-find run on a grid of pixels:



1. Every pixel is an element — an 8×10 image is an array of 80.



2. Call $\text{union}(p, q)$ on every pair of touching pixels that share a color.



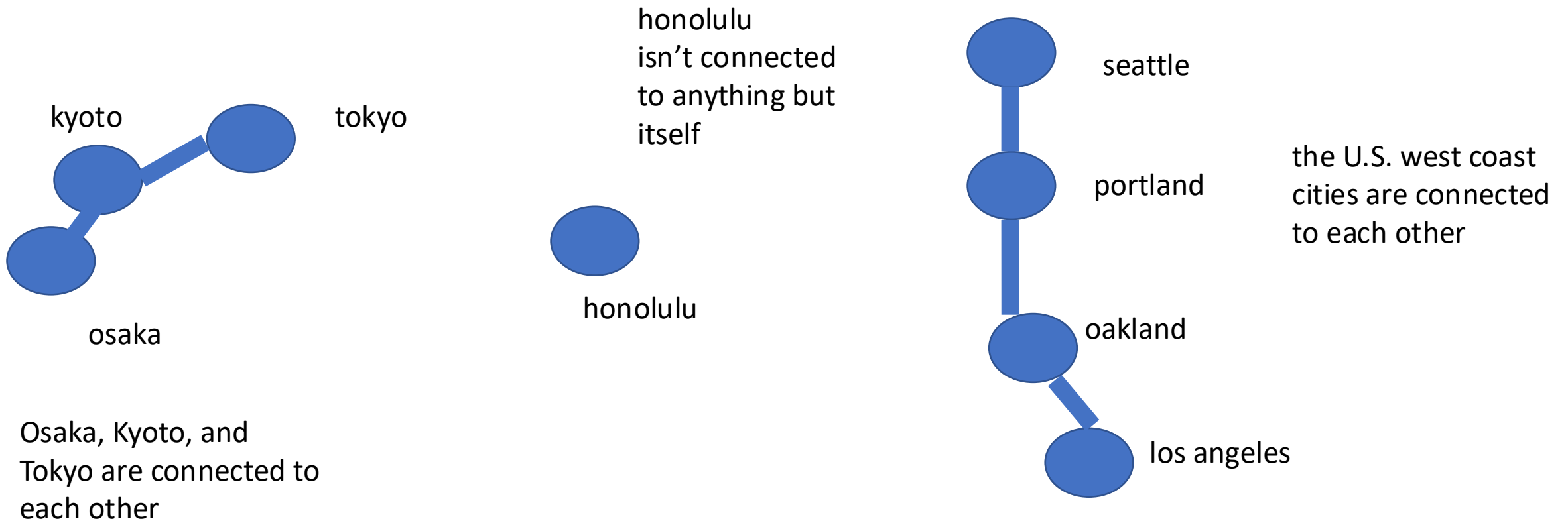
3. Click a pixel: $\text{find}()$ its root, then recolor every pixel with that same root.

The two blue shapes are the same color but sit in different components, so the fill stops at the gap. **Connectivity decides what gets painted, not color.**

So why union-find, and not just walk outward from the pixel you clicked? For one fill, walking is fine. But an editor does the unions **once**, and every click after that is a single $\text{find}()$ — we're basically storing the result of our calculation in a really efficient way

Here's the problem we want to solve:

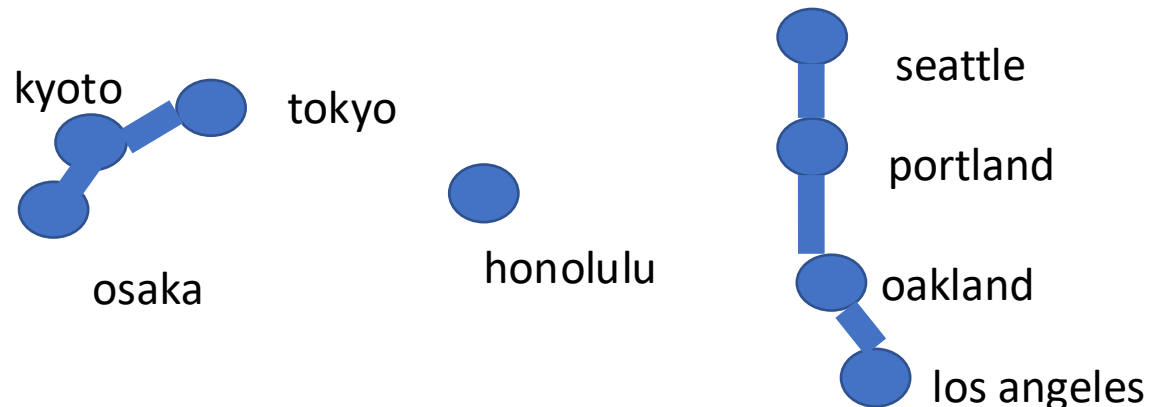
- If you want to make it slightly less abstract, you can think of nodes as cities and edges as highways between those locations



tangent: equivalence relation

- "is connected to" is what's called an *equivalence relation*. Something is an equivalency relation if all of the following are true:

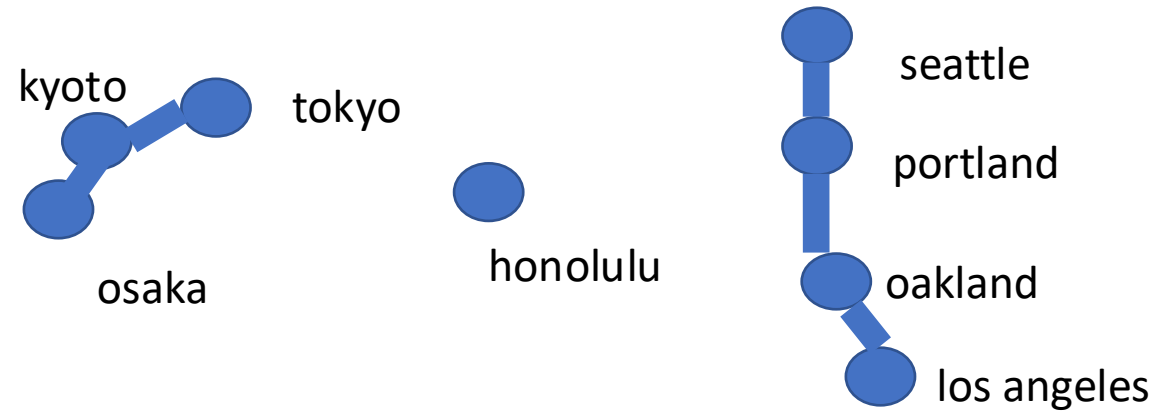
1. It's **reflexive**. Any element p is connected to itself (p is connected to p)
2. It's **symmetric**. If element p is connected to element q , then q is connected to p
3. It's **transitive**. If p is connected to q and q is connected to r , then p is connected to r .



1. portland is connected to portland
2. seattle is connected to portland, therefore portland is connected to seattle
3. oakland is connected to seattle through portland

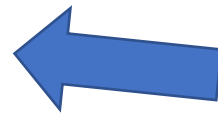
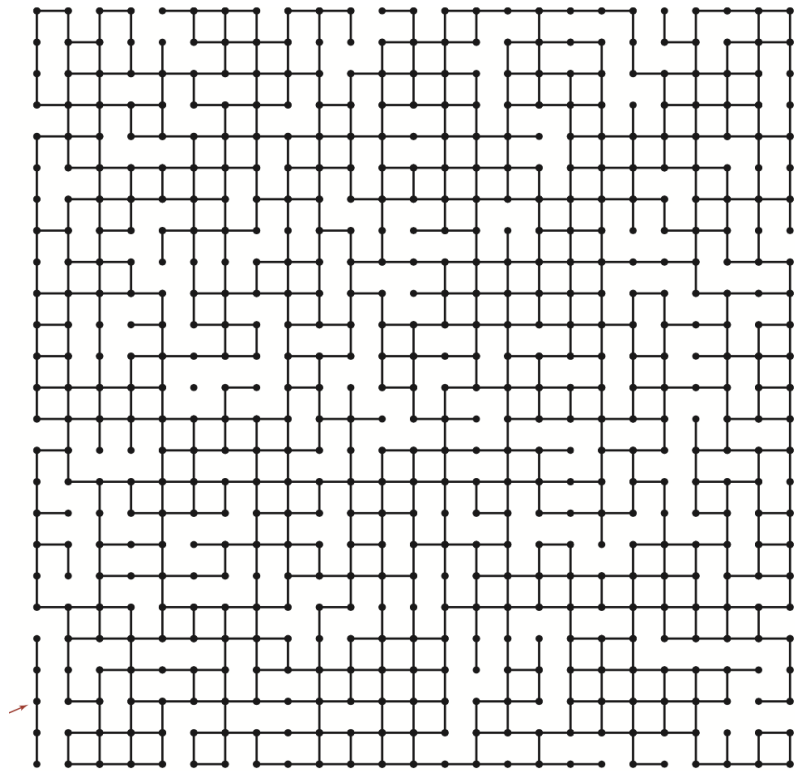
connected components

This set of elements has three *connected components*: the west coast cities are one, the Japanese cities are another, and Honolulu is a third.



A more complicated set of elements

- It's easy for a human to look at our cities graph and tell what elements are connected. But what if we had something like this?



according to the textbook
there are three connected
components here... but don't
ask me what they are...

END UNION-FIND PROBLEM STATEMENT
VIDEO

So here's what we want to do:

- We want to design an *abstract data type* that can:
 - Store a set of elements
 - Add connections between those elements, while still accurately reporting which components are connected
 - Tell us whether any two elements are in the same connected components.

We're going to need to implement the following functions:

$\text{union}(p, q)$	$\rightarrow$ add a connection between element p and element q
$\text{find}(p)$	$\rightarrow$ report info on what connected component p is in
$\text{connected}(p, q)$	$\rightarrow$ tell us whether p and q are connected

(this is pseudocode, not code. We're reasoning about this problem, not implementing a solution yet)

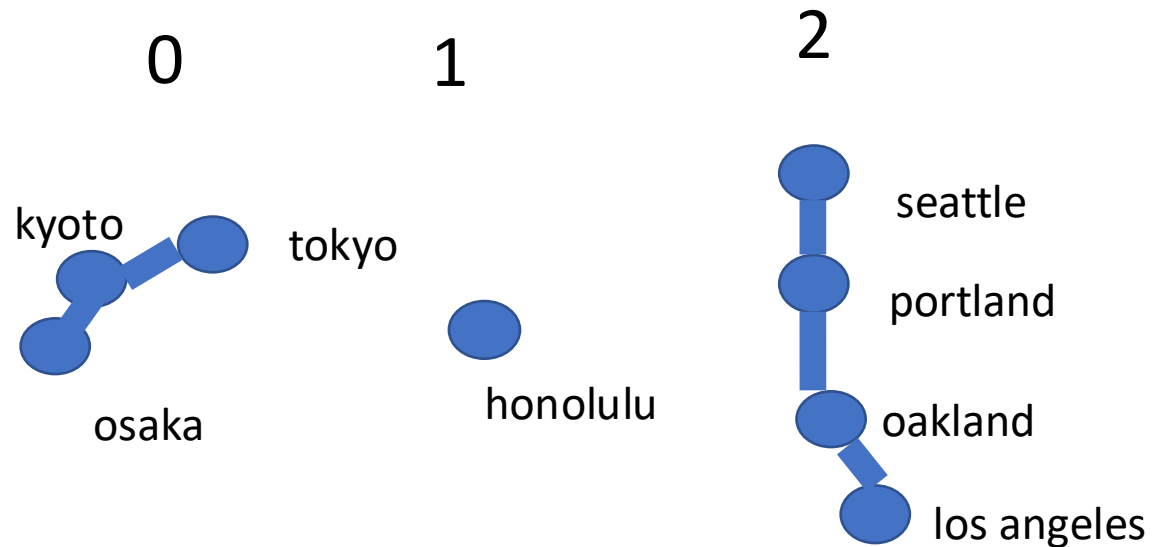
$\text{find}(p)$

Let's (arbitrarily) number our components.

If we tried **$\text{find}(\text{"oakland"})$** , our function would return 2

$\text{find}(\text{"tokyo"})$ would return 0

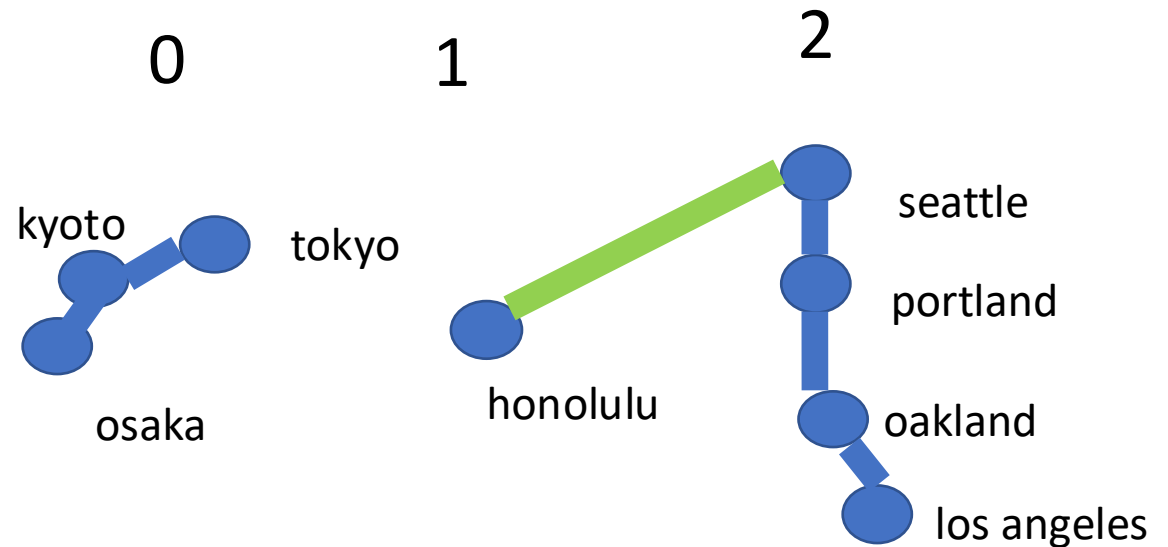
and so forth



$\text{union}(p, q)$

Our `union()` function adds a new connection between elements.

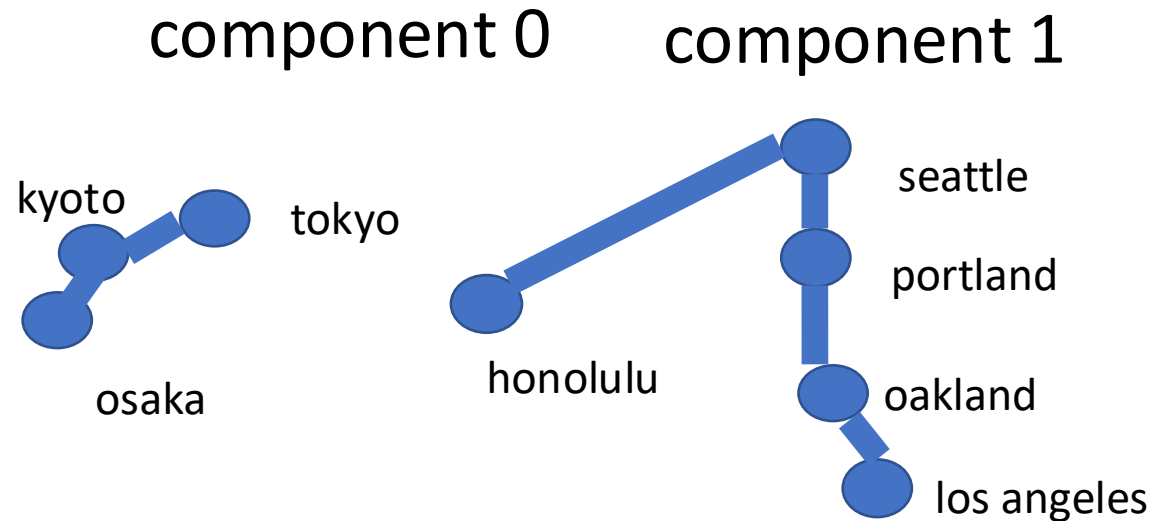
Let's say someone builds a magical bridge from Seattle to Hawaii: we would need to call **`union("seattle", "honorlulu")`**



$\text{union}(p, q)$

Now that the west coast cities and Honolulu are in the same connected component, we need to update the numbers we use to designate components

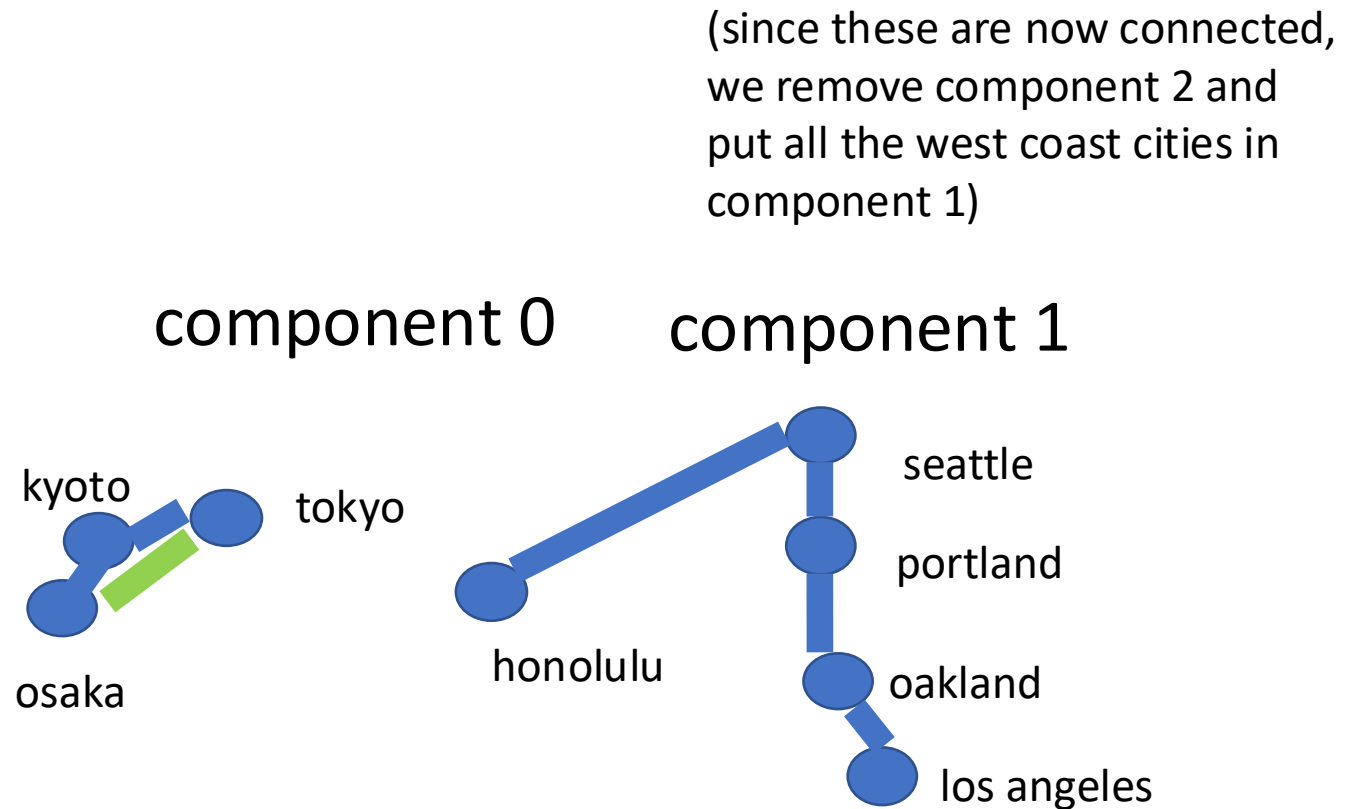
(since these are now connected, we remove component 2 and put all the west coast cities in component 1)



$\text{union}(p, q)$

If we add a connection between two elements that are already connected, we don't really have to do anything – all we care about is *whether elements are connected*, not about how many connections there are between elements or whether there's a direct path from one element to another.

If someone adds a direct connection between Tokyo and Osaka, it doesn't make a difference in terms of our connected components

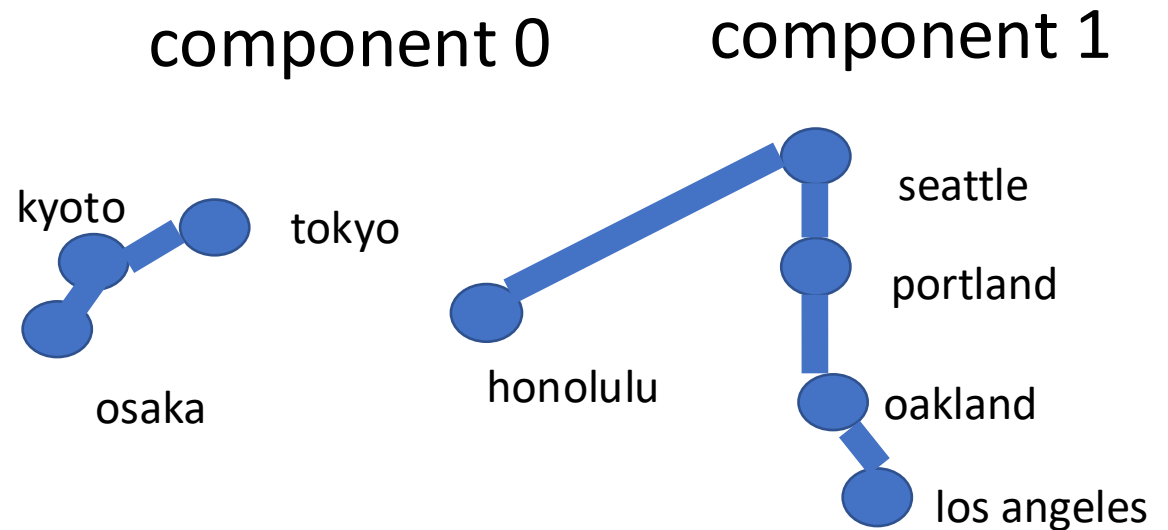


connected(p, q)

- connected(p, q) is pretty easy – we just call find(p) and find(q), and if and only if the values are the same, we return true.

connected("los angeles", "portland") -> **true**

connected("kyoto", "honolulu") -> **false**

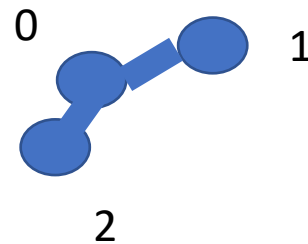


Back to numbers...

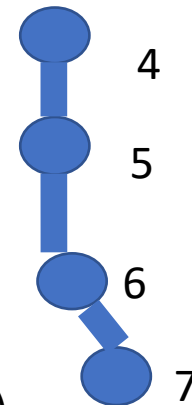
For the sake of convenience, let's stop using names for our elements and use numbers instead, like the textbook does it:

Why is this convenient? Because it makes storing data about our elements very straightforward. (see next slide)

component 0



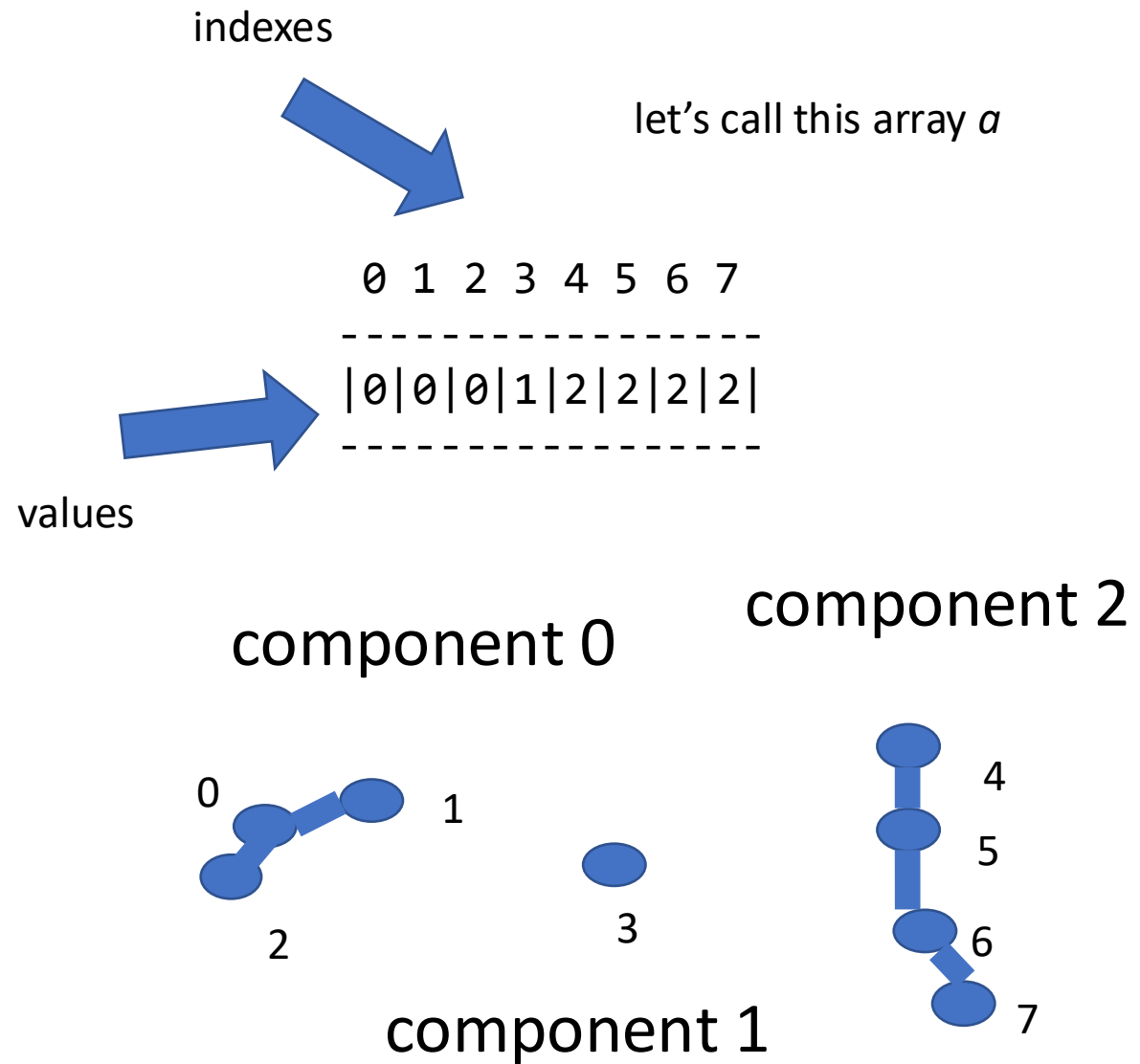
component 1



(let's also remove our bridge to hawaii...)

Let's put 'em in an array

- We'll store our data this way:
 - 1) Create array of size n . Each array index from 0 to $n-1$ corresponds to one of the elements in our set.
 - 2) Store in the array *the number of the component that each element is in.*

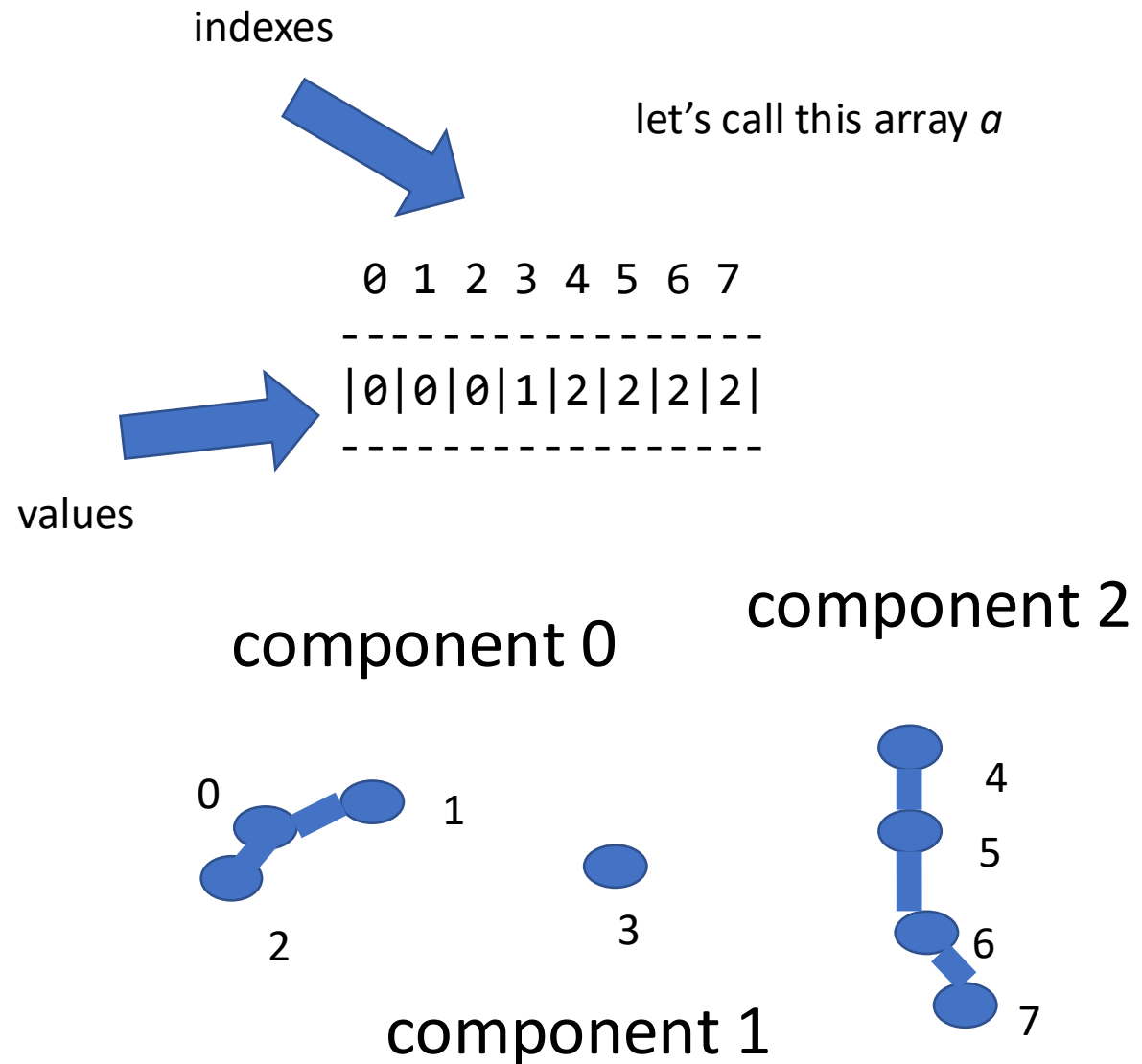


algorithm 1: quick-find

- What we're implementing here is the *quick-find* algorithm for union-find. Why is it called quick-find? because finding the component an element is in is very quick!

```
int find(int p){  
    return a[p];  
}
```

the value stored at index p tells us which connected component p is in

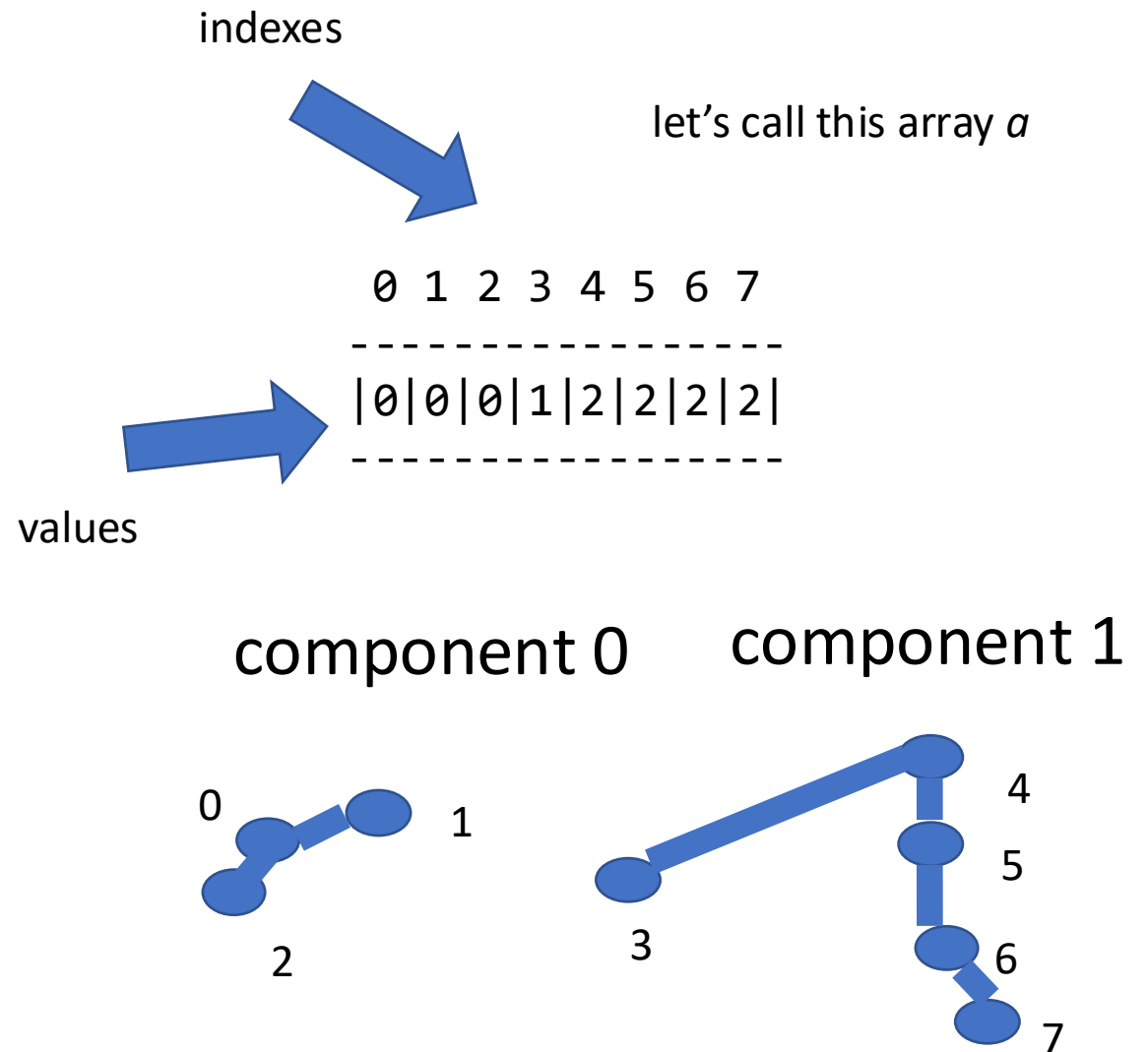


algorithm 1: quick-find

Let's think about how to do the union operation in this quick-find implementation, though.

Let's try adding back in our bridge to hawaii:

union(3, 4)

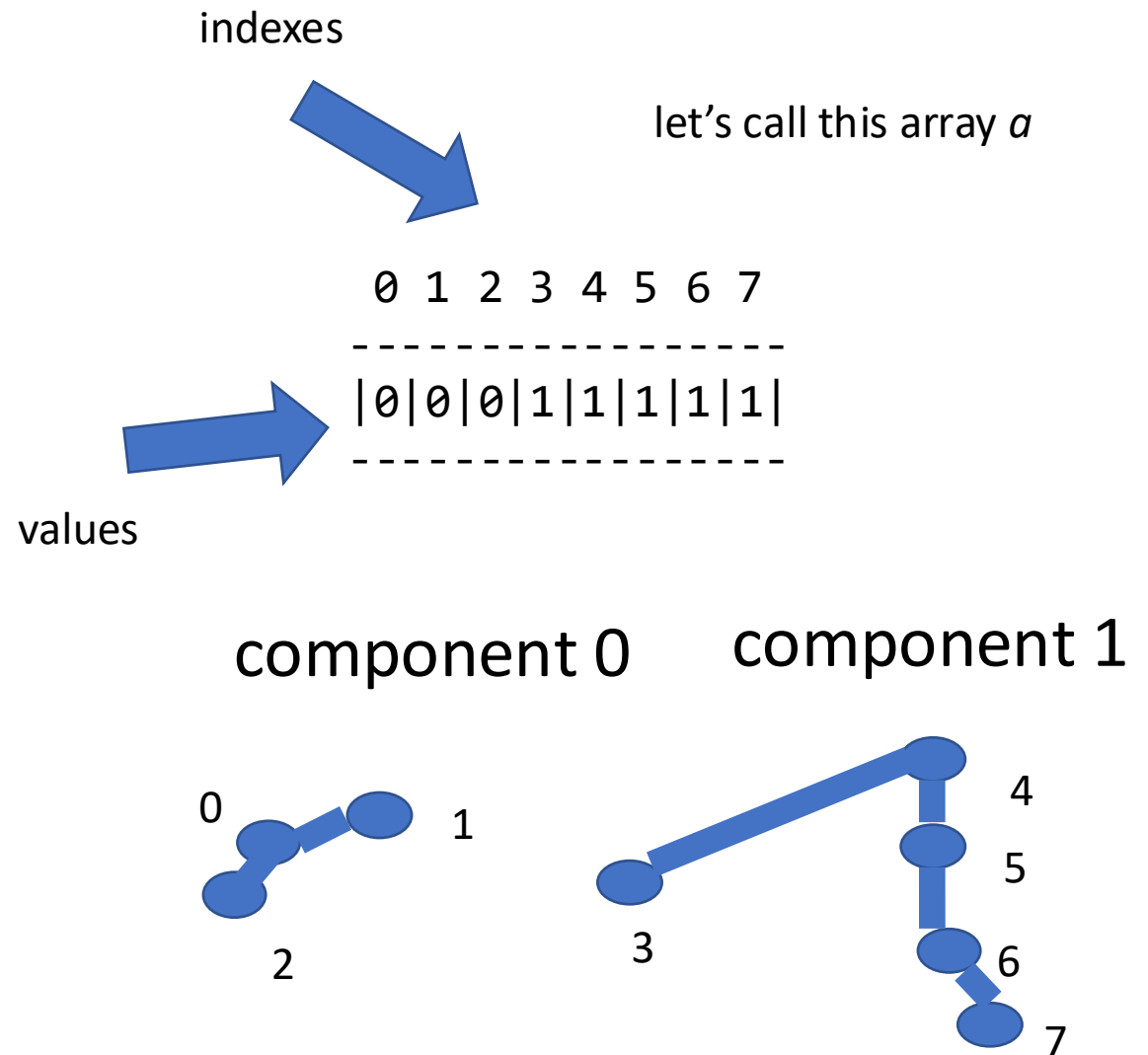


algorithm 1: quick-find

Let's try adding back in our bridge to hawaii:

union(3, 4)

We're now back down to two connected components, and we need to update our array accordingly... which means we have to scan through the array and change every 2 to 1

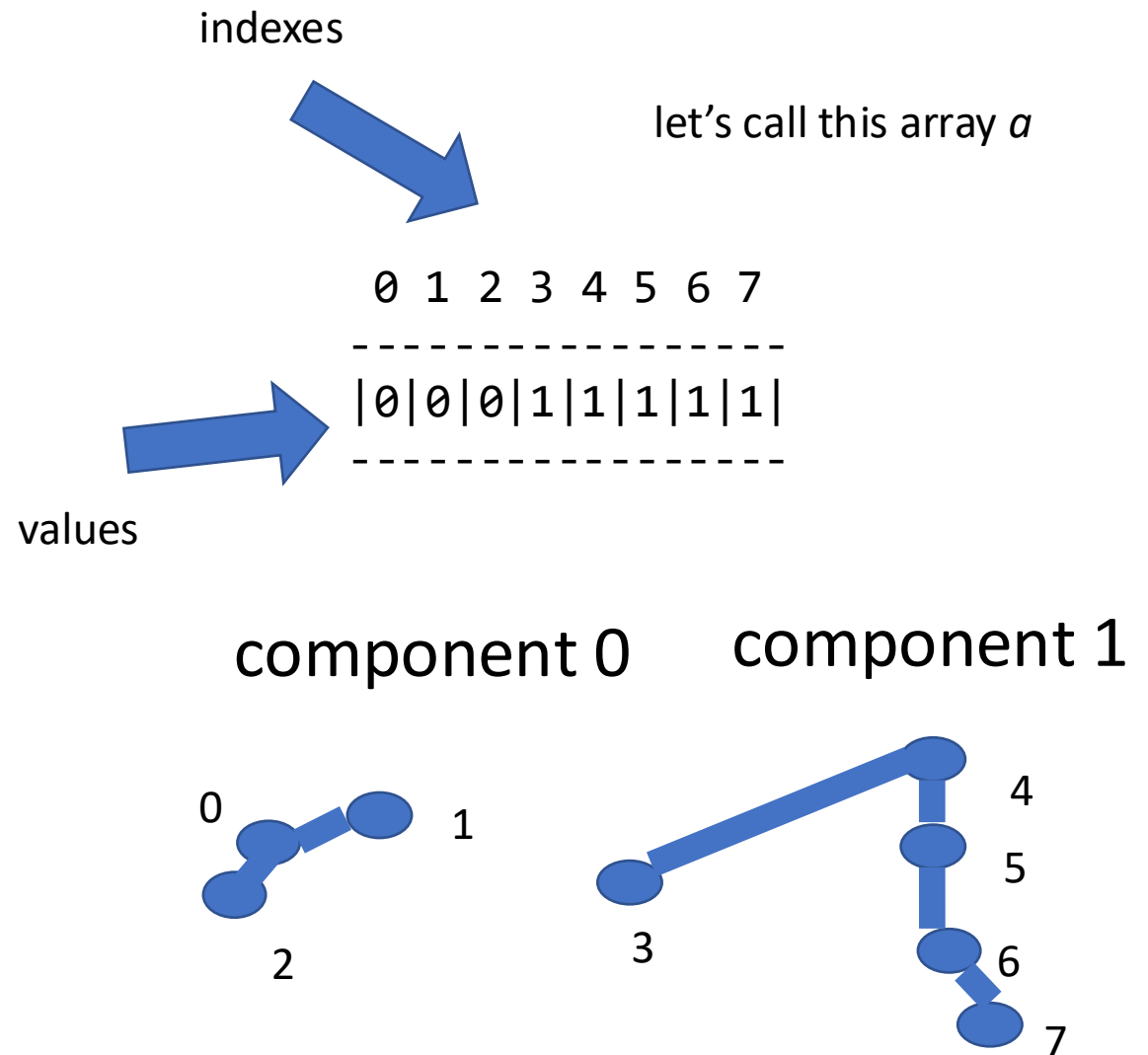


algorithm 1: quick-find

We're now back down to two connected components, and we need to update our array accordingly... which means we have to scan through the array and change every 2 to 1.

This is not particularly time-consuming on our small set of elements – we only have to look at 8 indexes, and if we find a 2, change it to 1.

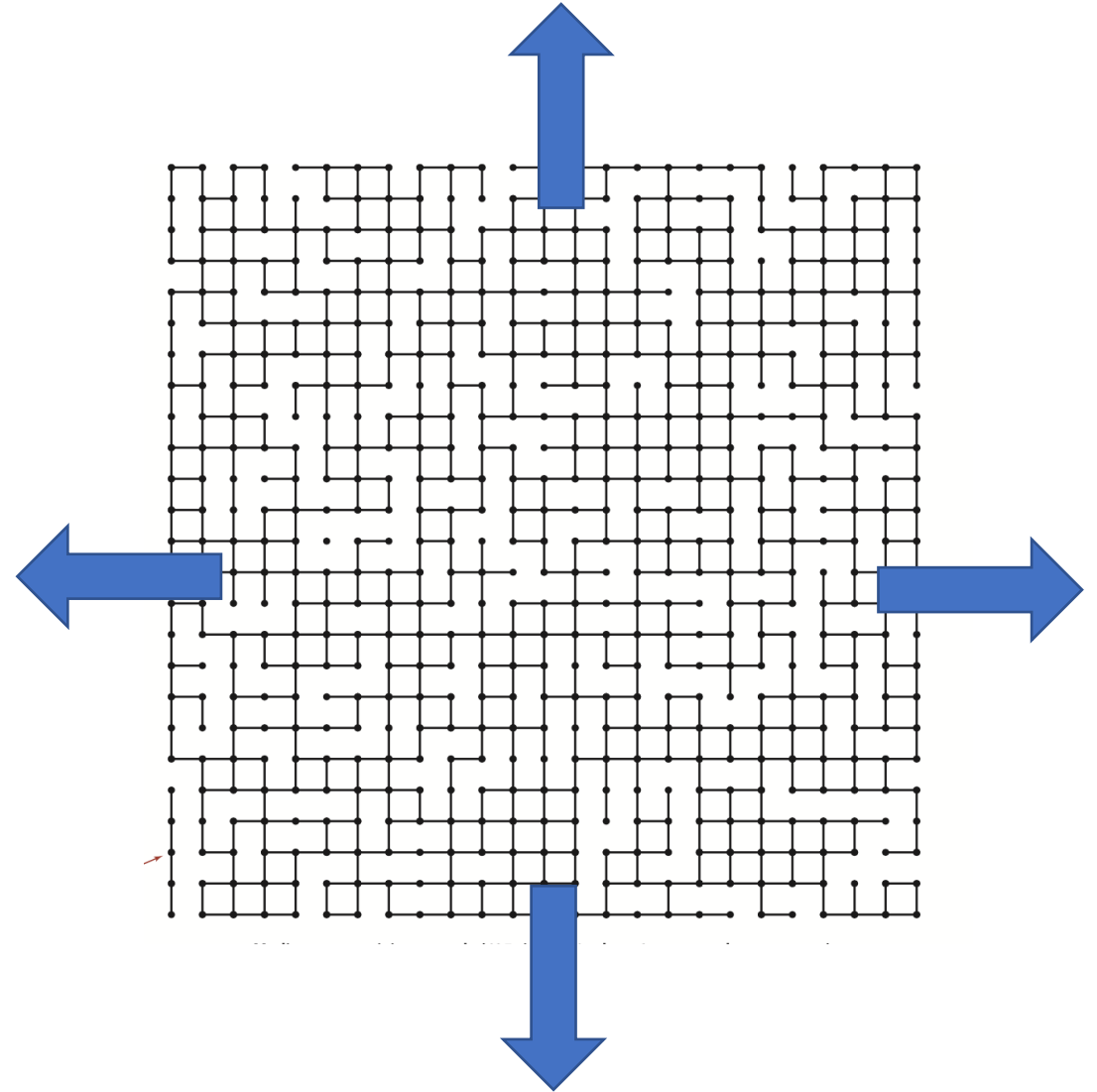
Let's pretend that each array access takes 10 microsecond – the union operation here only takes 80 microseconds!



algorithm 1: quick-find

But let's pretend we have a very large data set – much, much larger than the one on the right here. Let's say we've got a data set of 10 billion elements. (this is not implausible for some applications of union-find)

If each array access takes 10 microseconds, this will take 100,000 seconds – a bit over 28 hours. We do *not* want to have to spend 28 hours every time we do the union operation, especially since we might be doing the union operation quite often.



Can we do better?

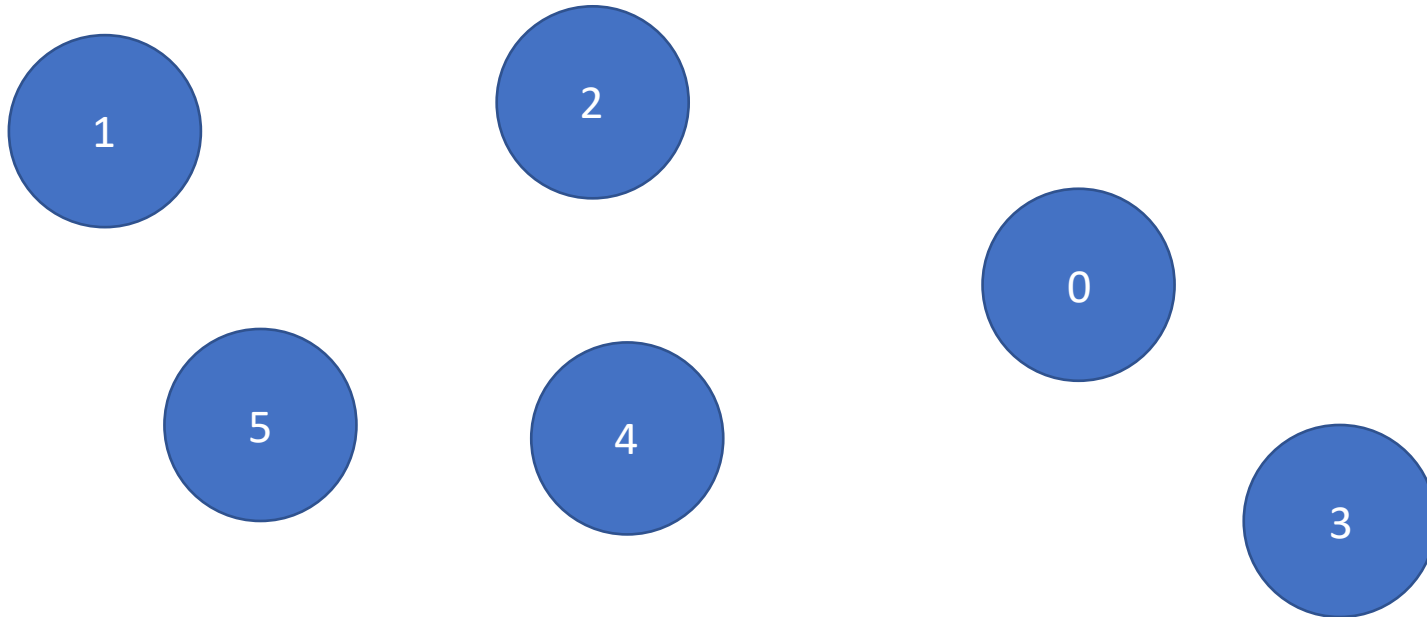
- Whenever you're doing algorithm design, you continually find yourself asking: *can we do better?*

Attempt at doing better : quick-union

- This attempt is based on the same data structure as quick-find – each element in the set is an index in an array.
- However, what gets stored in the array is different than in quick-find.
- Instead of storing the number of the connected component that the element is in, the value stored at an elements array index is *the number of another element connected to it*.

Attempt at doing better : quick-union

- Consider this set of 6 initially unconnected elements:

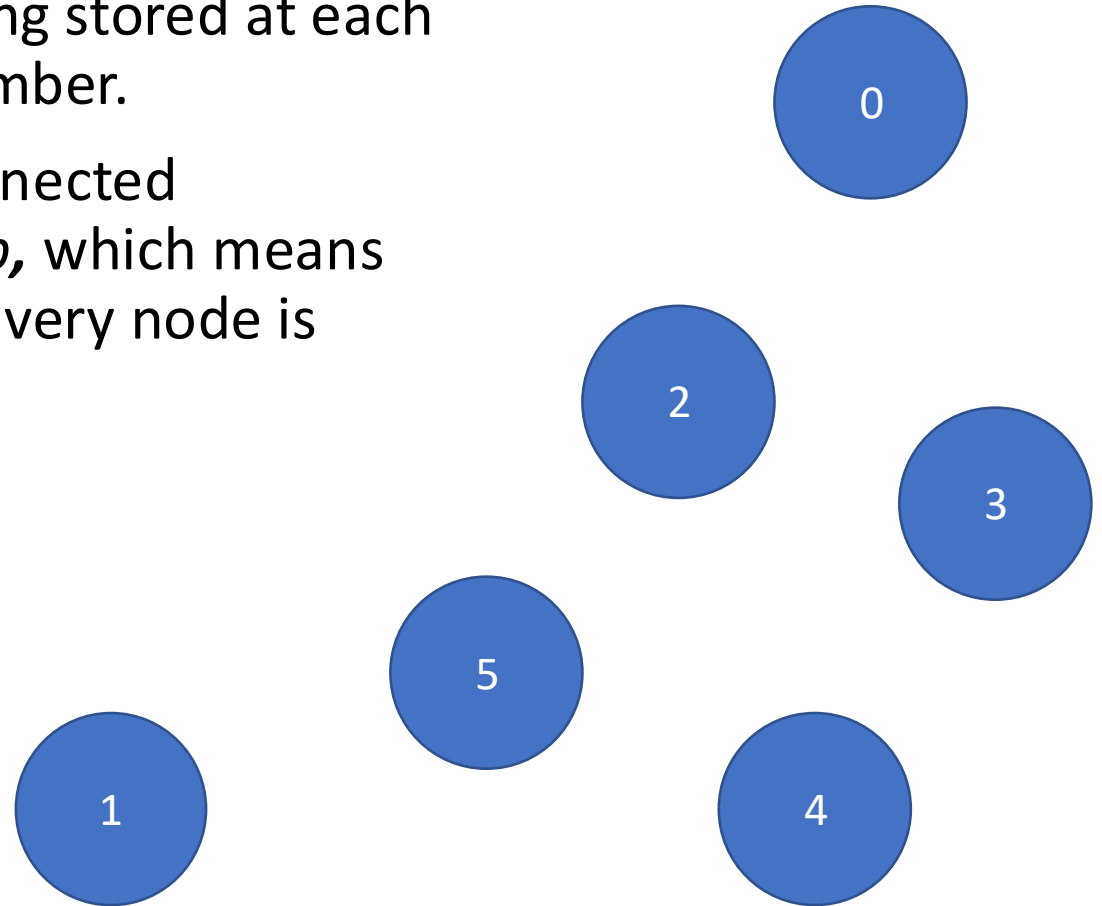


Attempt at doing better : quick-union

- We initialize the array a such that the thing stored at each element's index is the element's own number.
- Why? Remember that belonging to a connected component is an *equivalence relationship*, which means that (among other things) it is *reflexive*: every node is connected to itself.

0	1	2	3	4	5

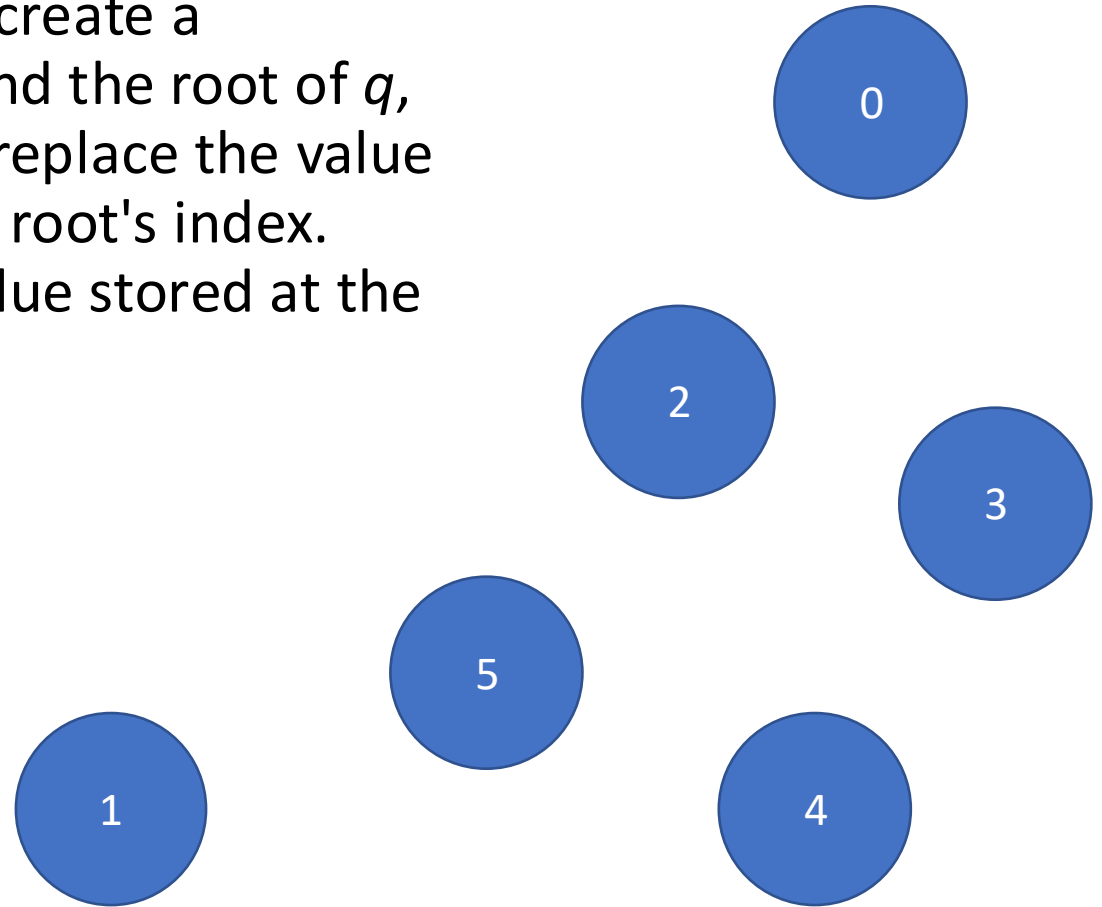
0	1	2	3	4	5



Attempt at doing better : quick-union

- When we use the `union(p, q)` method to create a connection, we first *find()* the root of p and the root of q , then choose one of those two roots and replace the value stored at *that root's* index with the other root's index. Let's (arbitrarily) decide to change the value stored at the root of p 's index

```
  0 1 2 3 4 5
-----
|0|1|2|3|4|5|
-----
```

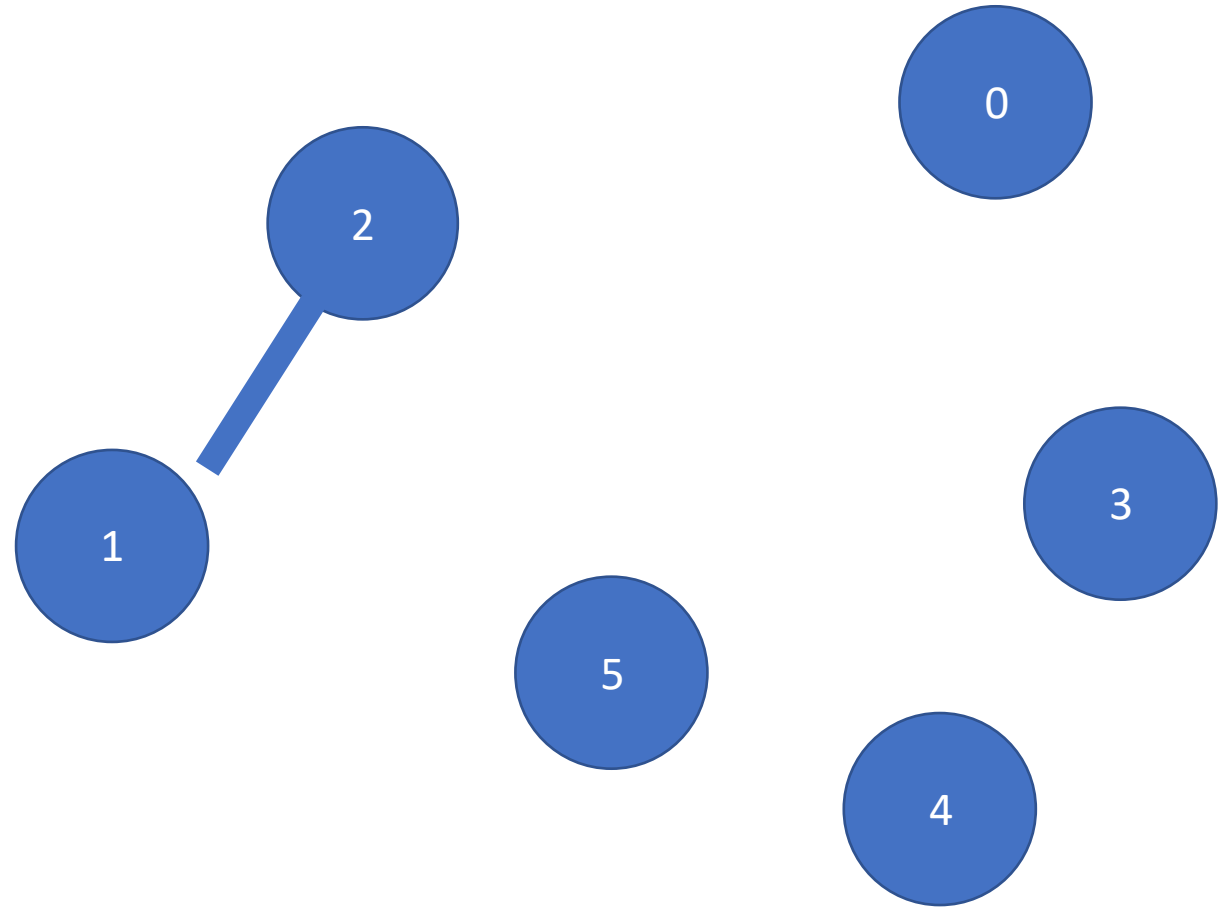


Attempt at doing better : quick-union

- **union(1, 2)**

0	1	2	3	4	5

0	2	2	3	4	5

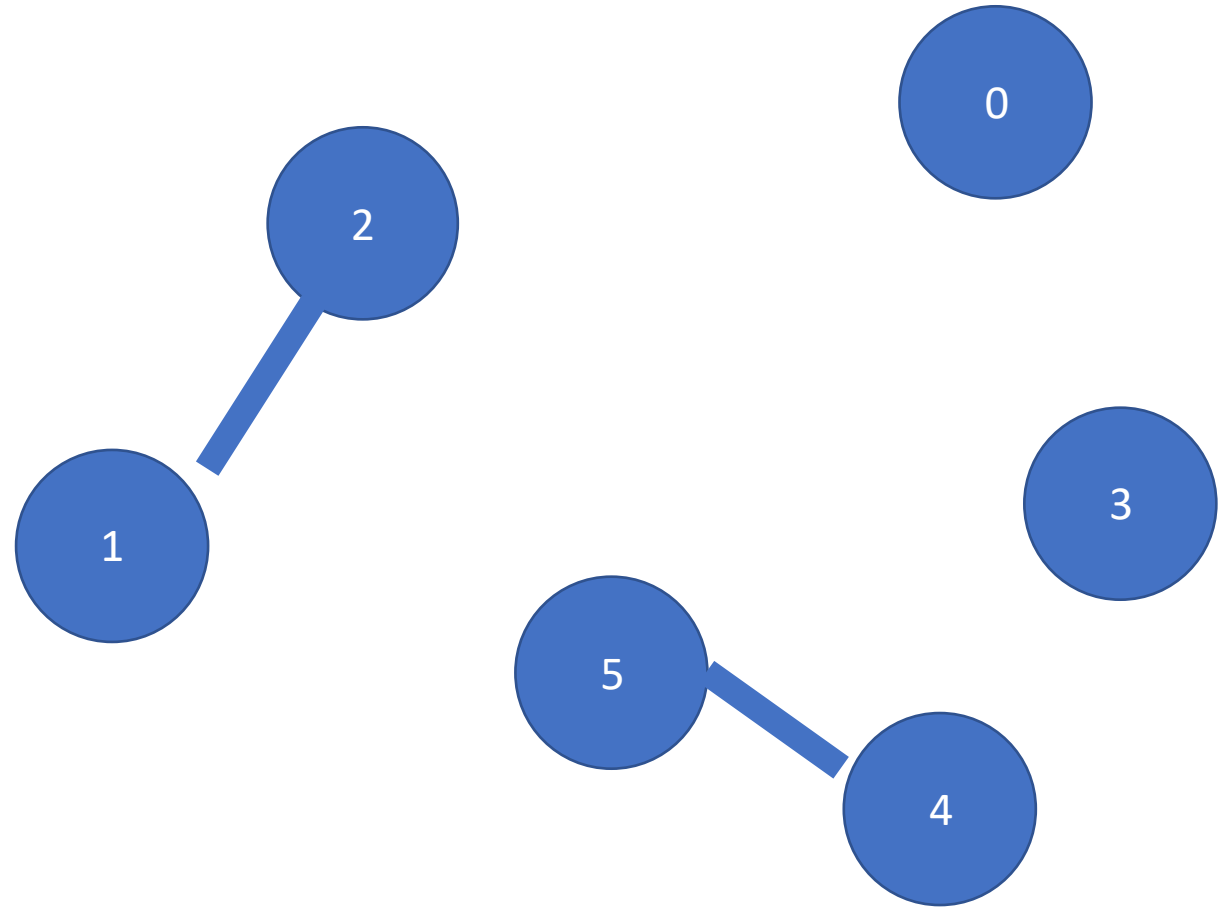


Attempt at doing better : quick-union

- **union(1, 2)**
- **union(4, 5)**

0	1	2	3	4	5

0	2	2	3	5	5

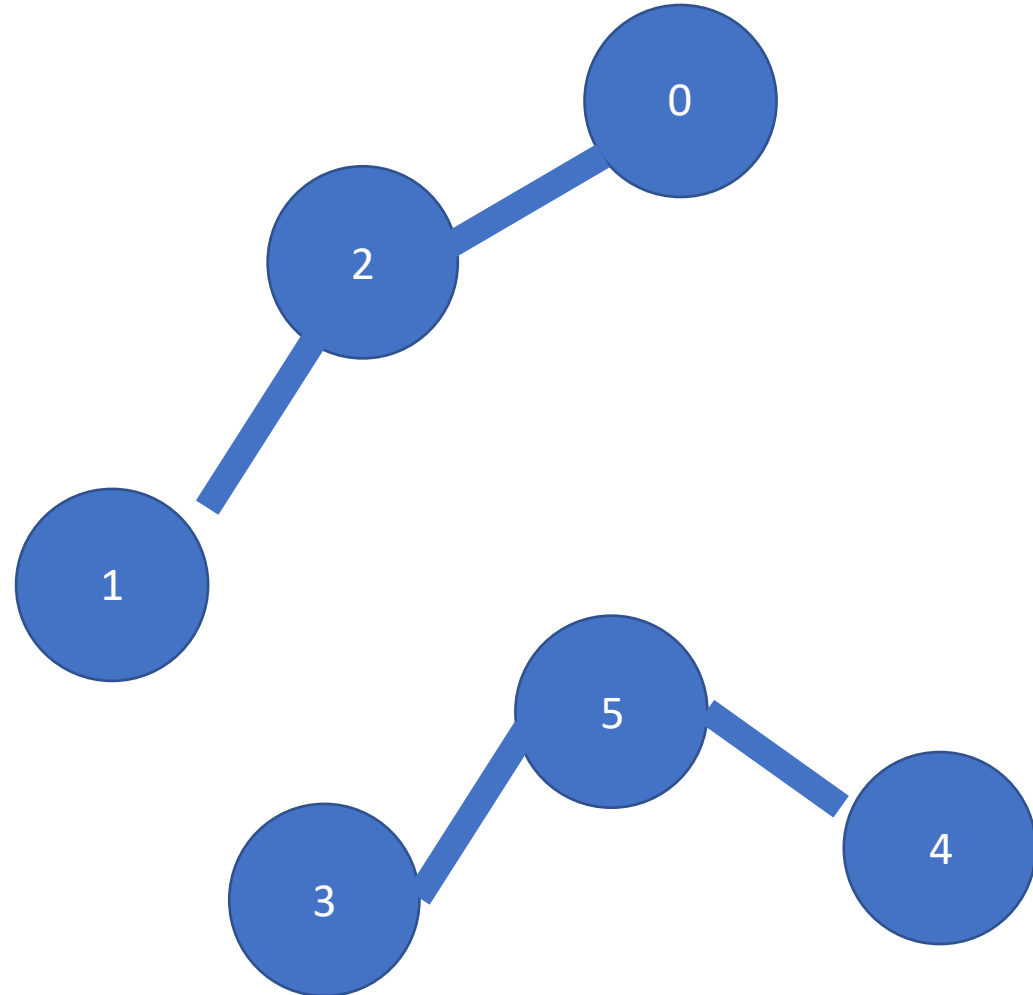


Attempt at doing better : quick-union

- **union(1, 2)**
- **union(4, 5)**
- **union(3, 5)**
- **union(2, 0)**

0	1	2	3	4	5

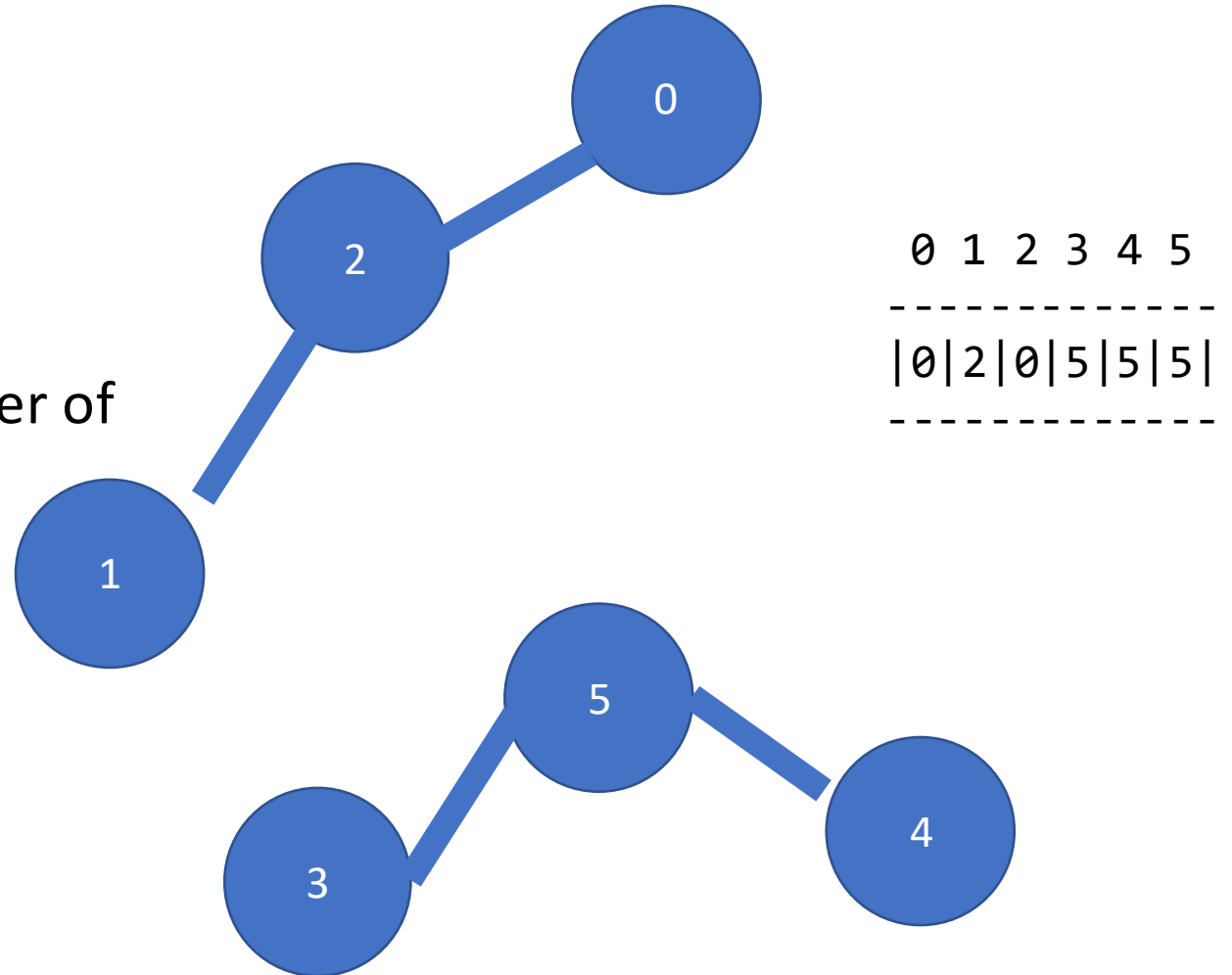
0	2	0	5	5	5



Attempt at doing better : quick-union

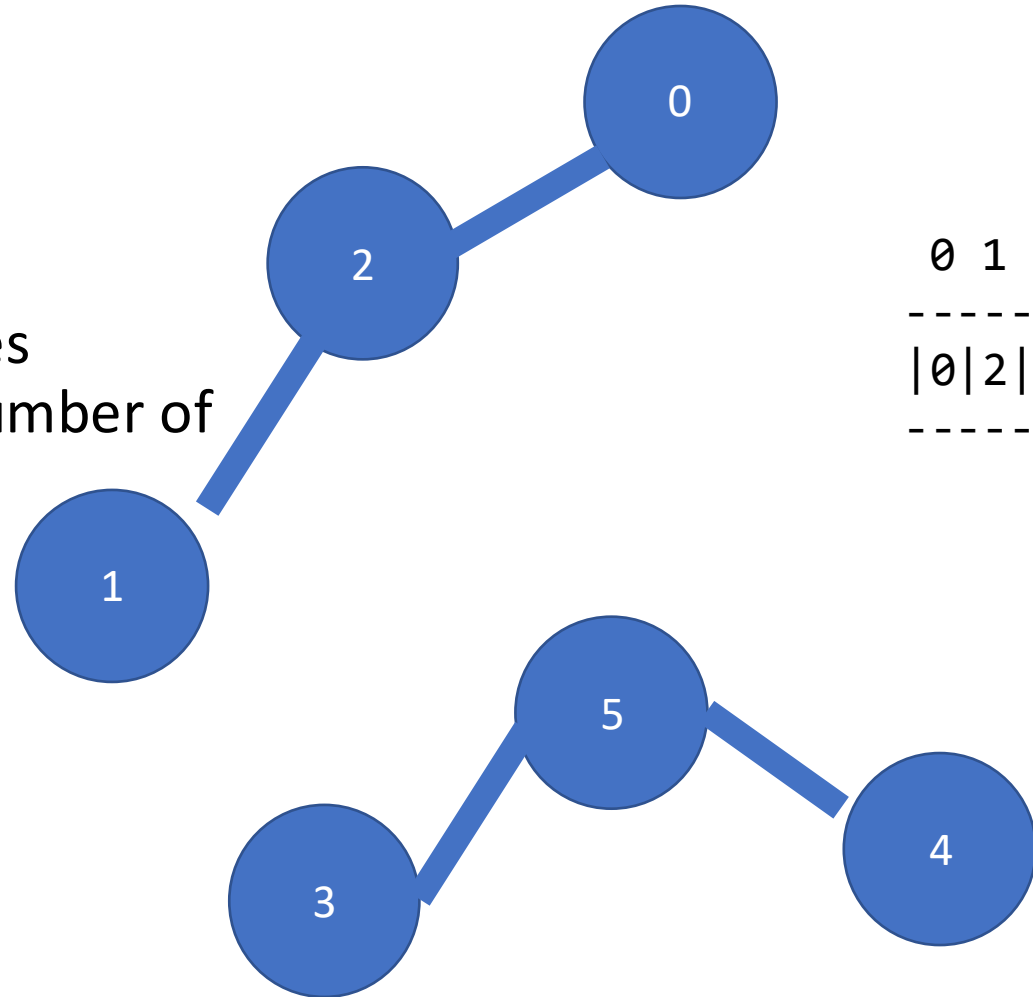
- Okay, so how do we do **find(p)**?
- The process:
 - 1) Check and see if the element stores its own number. If so, that's the number of its component.

find(0) -> 0



Attempt at doing better : quick-union

- Okay, so how do we do **find(p)**?
- The process:
 - 1) Check and see if the element stores its own number. If so, that's the number of its component.
 - 2) If it doesn't contain its own number, look at the element with the number it contains.



0	1	2	3	4	5

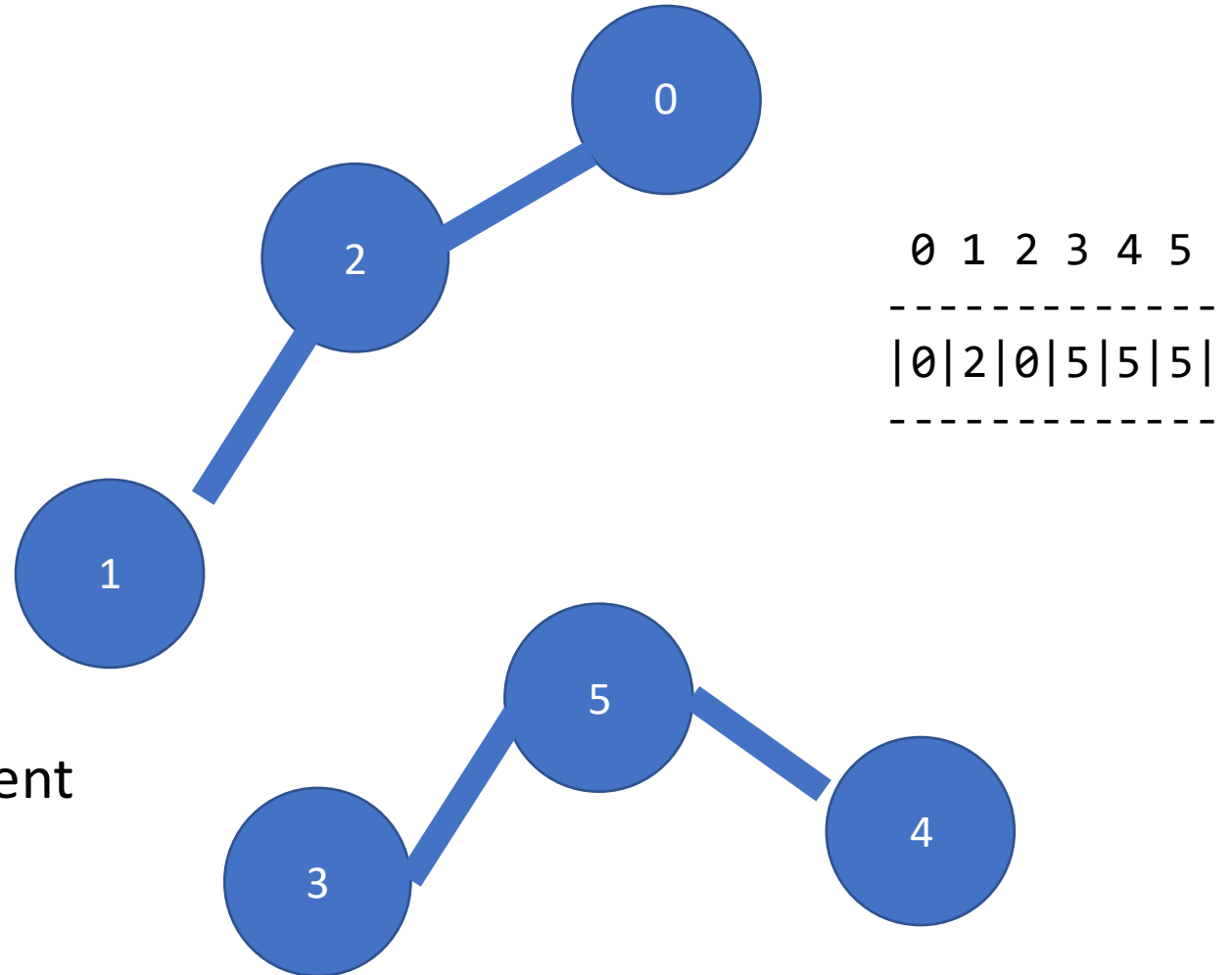
0	2	0	5	5	5

Attempt at doing better : quick-union

- Okay, so how do we do **find(p)**?

Let's try 3.

1. 3 contains the value 5.
2. We now check 5.
3. 5 contains the value 5 – its own value
4. **that's** the number of the component 3 is in.

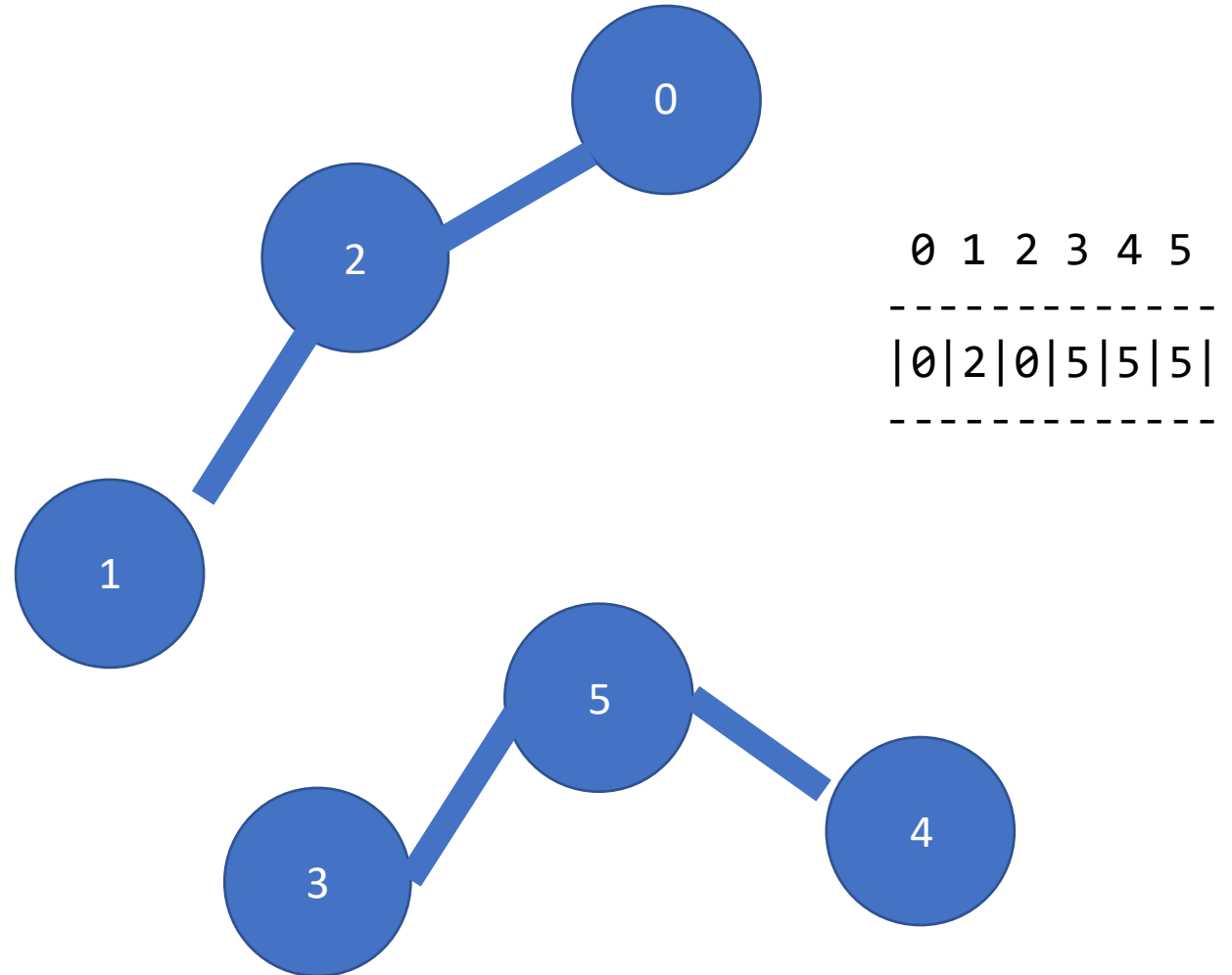


Attempt at doing better : quick-union

- Okay, so how do we do **find(p)**?

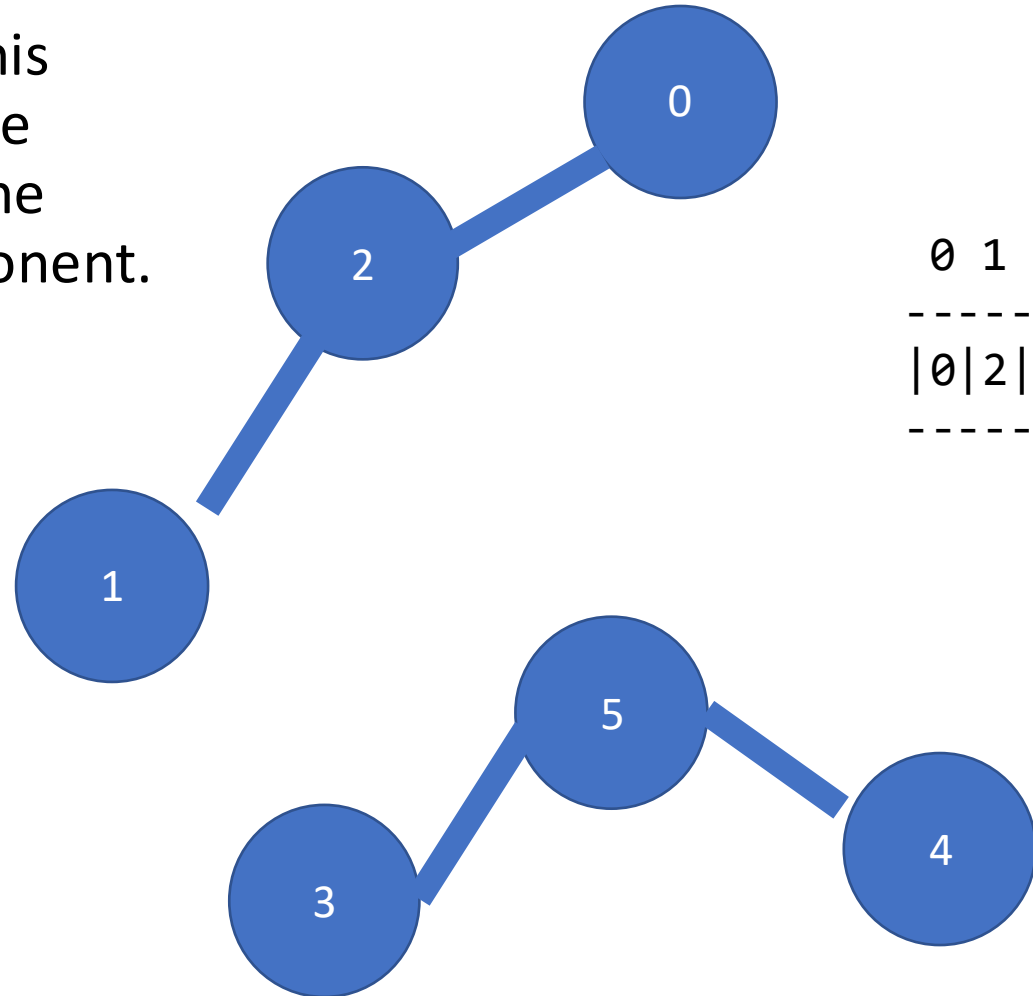
Let's try 1.

1. 1 contains the value 2
2. 2 contains the value 0
3. 0 contains the value 0.
4. **that's** the number of the component 1 is in.



Attempt at doing better : quick-union

The node we find when we perform this process we designate as the *root* of the tree. Any two nodes that have the same root are in the same connected component.



0	1	2	3	4	5

0	2	0	5	5	5

How much better is it?

- Well, union is definitely easier than in quick-find – we only have to change one number!
- However, find can *potentially* be very slow.
- Consider the case that we perform a set of connections such that our trees end up being very *tall*:

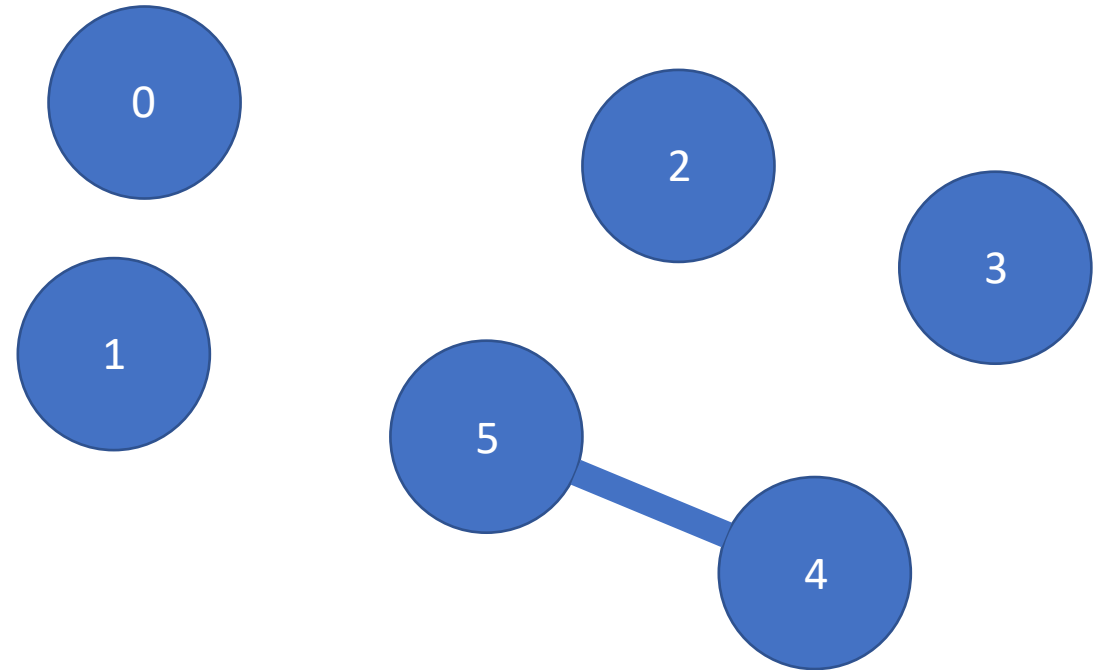
How much better is it?

- Consider the case that we perform a set of connections such that our trees end up being very *tall*:

`union(5, 4)`

0	1	2	3	4	5

0	1	2	3	4	4

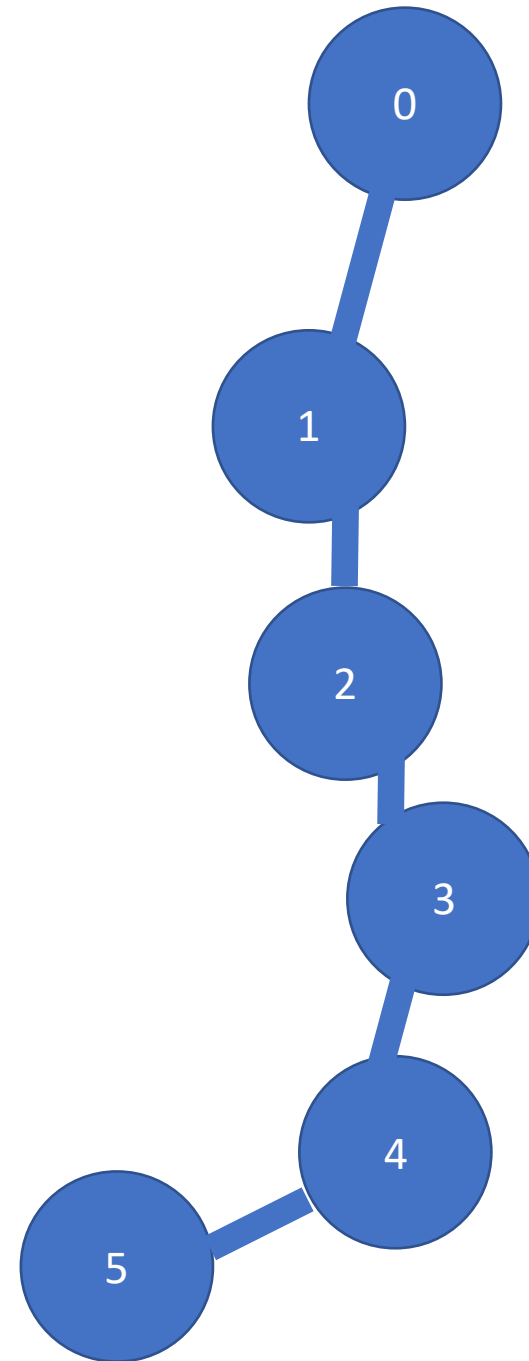


How much better is it?

- Consider the case that we perform a set of connections such that our trees end up being very *tall*:

`union(5, 4)`
`union(4, 3)`
`union(3, 2)`
`union(2, 1)`
`union(1, 0)`

0	1	2	3	4	5
0	0	1	2	3	4



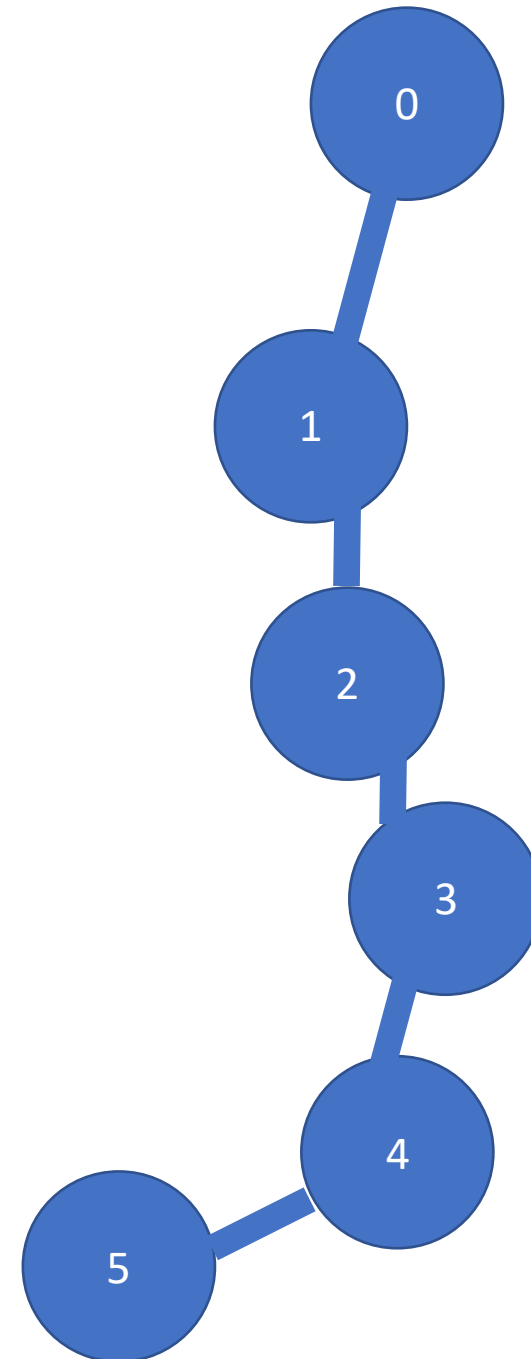
tall tree
is tall

How much better is it?

- There's only one component in this set, and its value is 0 – the value of the root of the tree.
- However, how long does it take to do **find(5)**?
- 5 goes to 4
- 4 goes to 3
- 3 goes to 2
- 2 goes to 1
- 1 goes to 0
- we found it...
... in five steps.

0	1	2	3	4	5

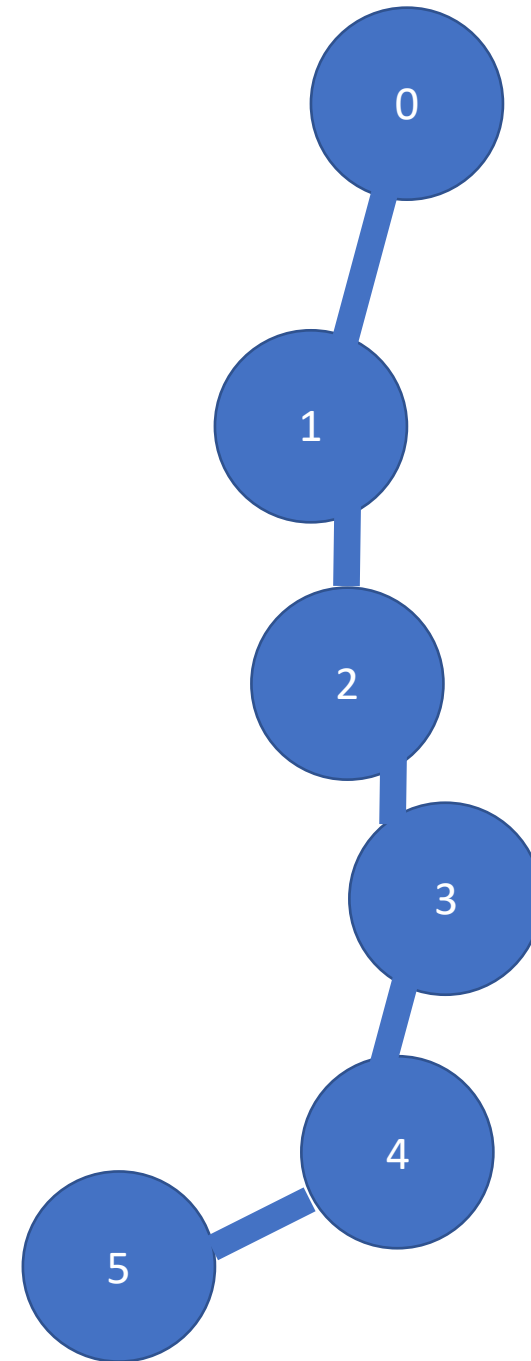
0	0	1	2	3	4



tall tree
is tall

How much better is it?

- This isn't so bad if we're only dealing with five items... but we're in a universe where we might be dealing with *billions* of items.
- We once again have a problem: whereas union was very slow in quick-find, find can potentially be very slow in quick-union.



tall tree
is tall

CAN WE DO BETTER?

(I promise you, you'll be *very* sick of this question
by the end of the semester)

END QUICK-FIND AND QUICK-UNION VIDEO

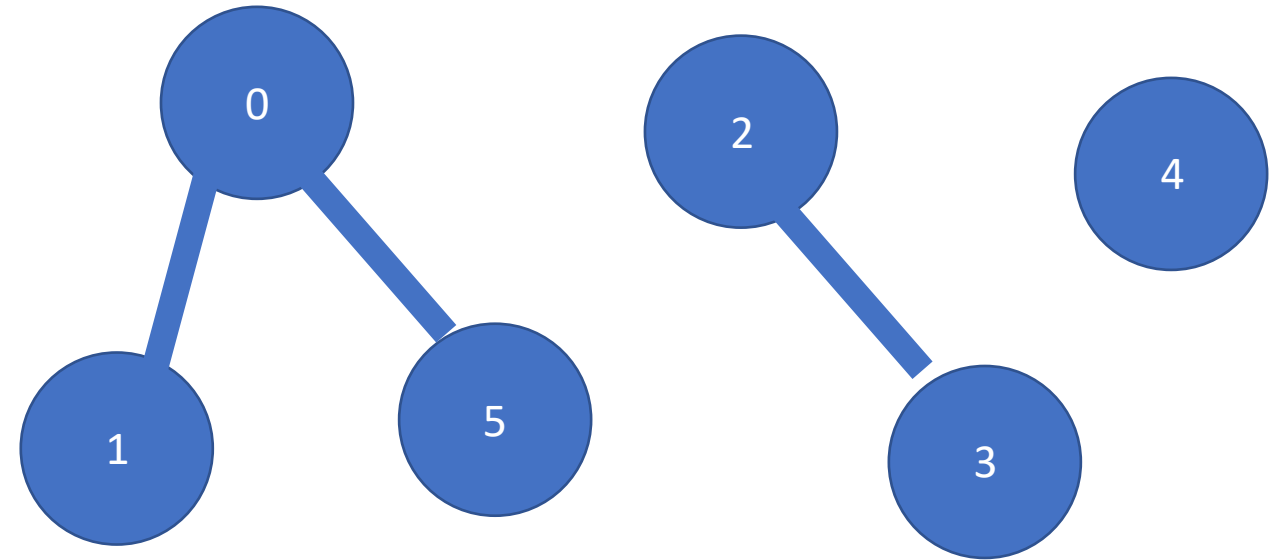
Weighted quick-union, or, making our trees happier and littler



- We want to avoid letting our trees get tall.
- Here's what we're going to do:
when we merge two trees, we *attach the root of the smaller tree to the root of the larger tree*

0	1	2	3	4	5

0	0	2	2	4	0



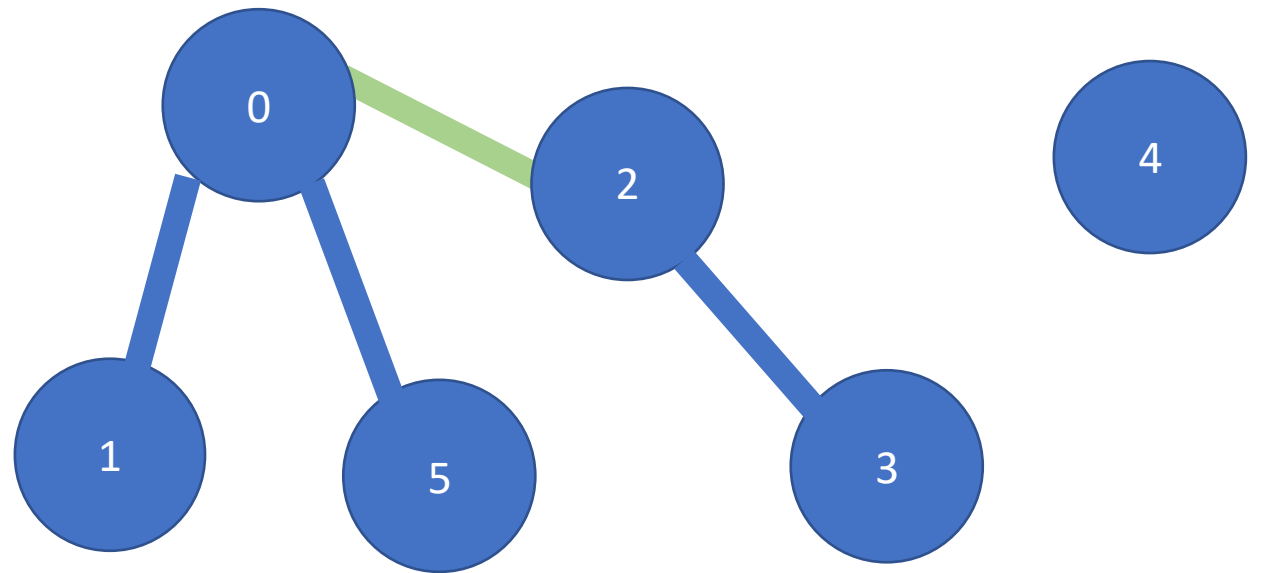
Weighted quick-union, or, making our trees happier and littler

- Here's what we're going to do: when we merge two trees, we *attach the root of the smaller tree to the root of the larger tree*.
- **union(3, 5)**

0	1	2	3	4	5

0	0	0	2	4	0

(note that we only update one value)

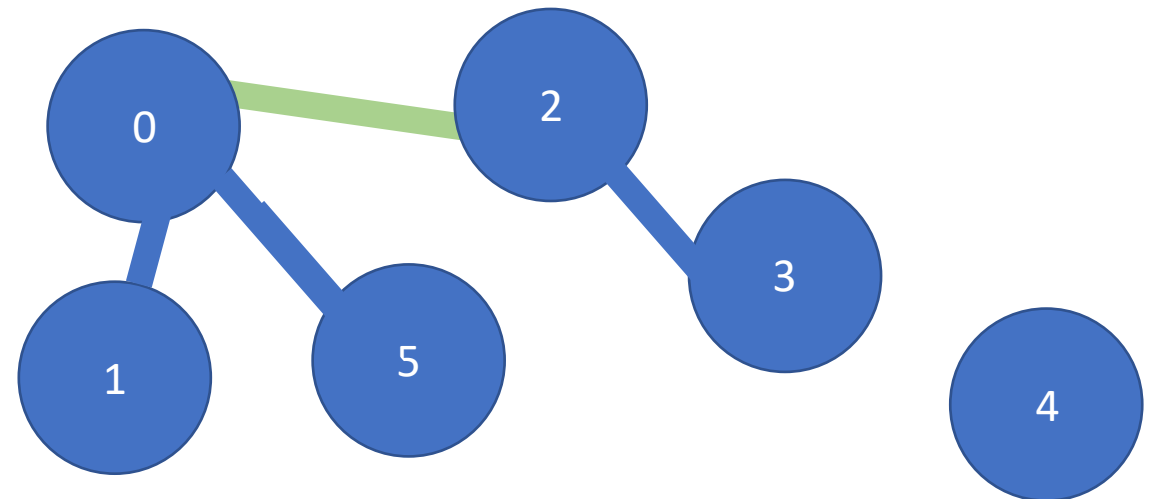


A wrinkle!

- We need to keep track of *the size of the tree that each node is in*
- this means a second array
- we update that array when we do a union operation

0	1	2	3	4	5

0	0	2	2	4	0



A wrinkle!

- We need to keep track of *the size of the tree that each node is in*
- this means a second array

of nodes in
the tree *rooted*
at this item

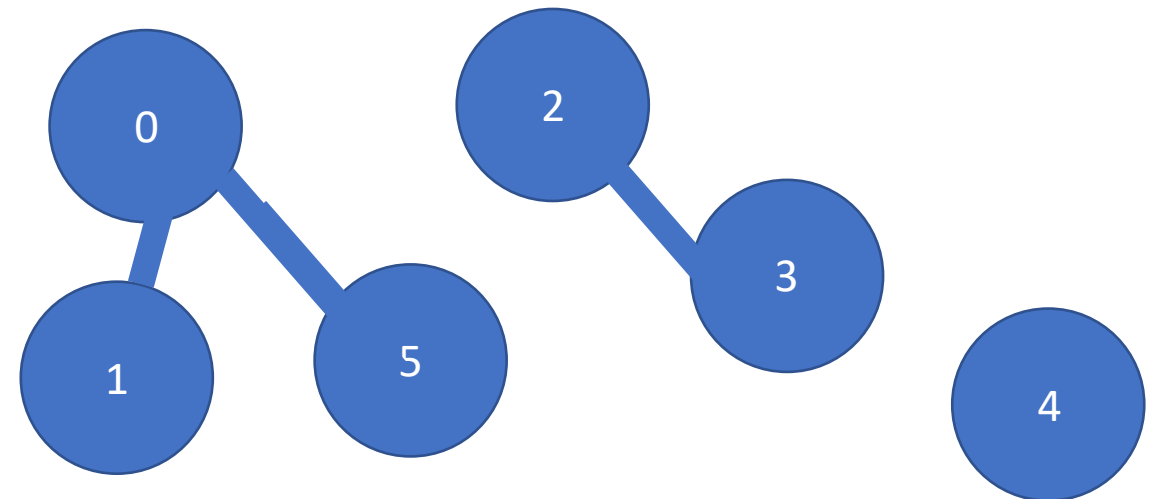


0	1	2	3	4	5

0	0	2	2	4	0

0	1	2	3	4	5

3	1	2	1	1	1



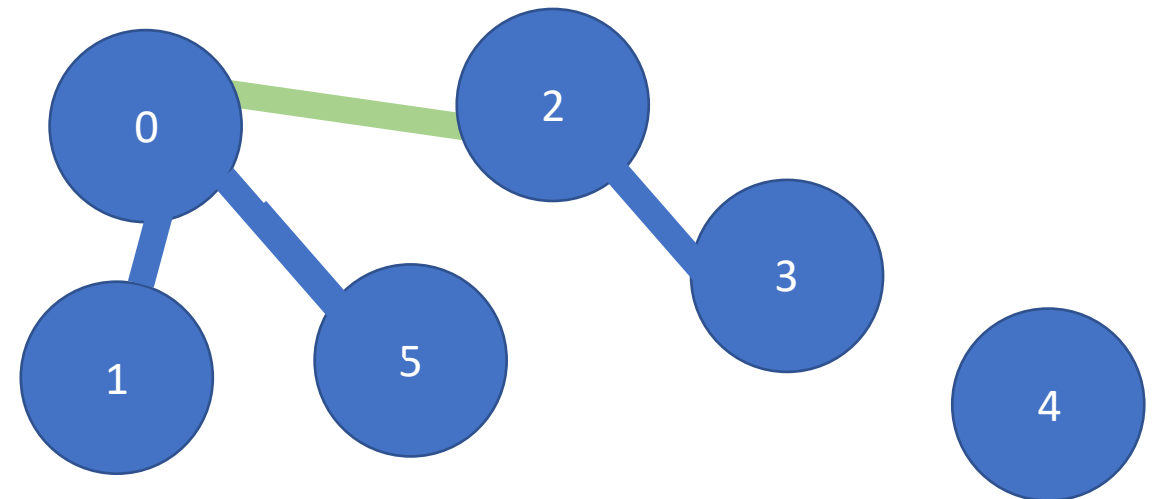
A wrinkle!

- We need to keep track of *the size of the tree that each node is in*
- this means a second array

set the value stored in
2 to 0

0	1	2	3	4	5
0	0	0	2	4	0

0	1	2	3	4	5
3	1	2	1	1	1



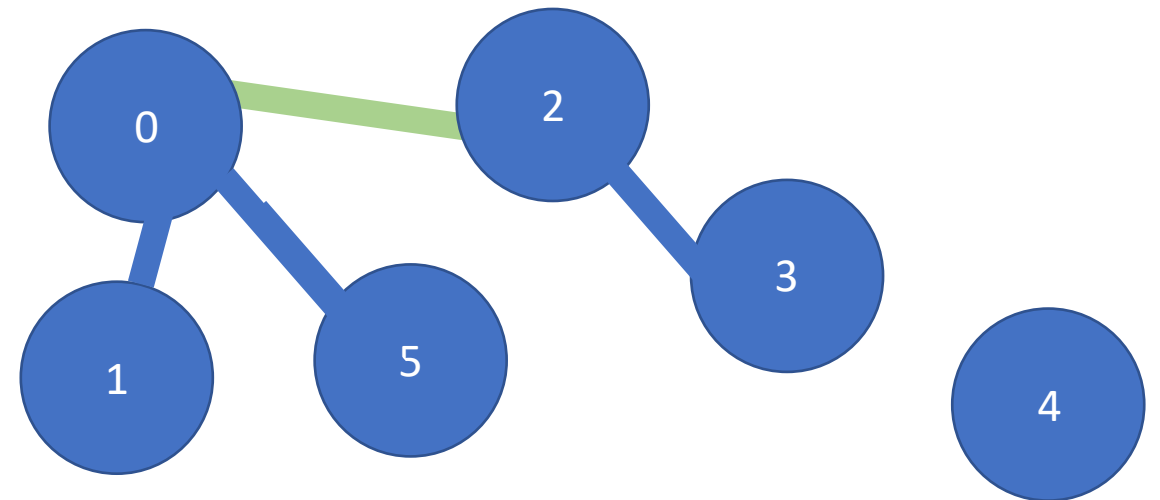
A wrinkle!

- We need to keep track of *the size of the tree that each node is in*
- this means a second array

0	1	2	3	4	5
0	0	0	2	4	0

0	1	2	3	4	5
5	1	2	1	1	1

Set the value stored in
0 in our size array to 5



CAN WE DO EVEN BETTER?

(I promise you, you'll be *very* sick of this question
by the end of the semester)

How good is weighted quick-union?

- Every union() attaches the smaller tree under the root of the larger one.
- That means a tree's height can *only* grow when we merge two trees that happen to be the same size — sliding a small tree under a big one never makes the big one any taller.

- **So every time the height climbs by 1, the tree's size has to at least double:**

height	0	1	2	3	4	5	...	30
min size	1	2	4	8	16	32	...	~1 billion

even with a billion elements, the tree is at most 30 levels deep

"logarithmic time"

- This runs in "logarithmic time" – we'll talk about what this means more later in the semester
- The short version: starting from 1, $\log_2(n)$ is the number of times you have to double before reaching n
- $\log_2(8)$ is 3, $\log_2(16)$ is 4, $\log_2(1,000,000,000)$ is 30.
- The $\log_2(n)$ grows *very slowly*. This is good!

height	0	1	2	3	4	5	...	30
min size	1	2	4	8	16	32	...	~1 billion

even with a billion elements, the tree is at most 30 levels deep

Can we do even better? Path compression

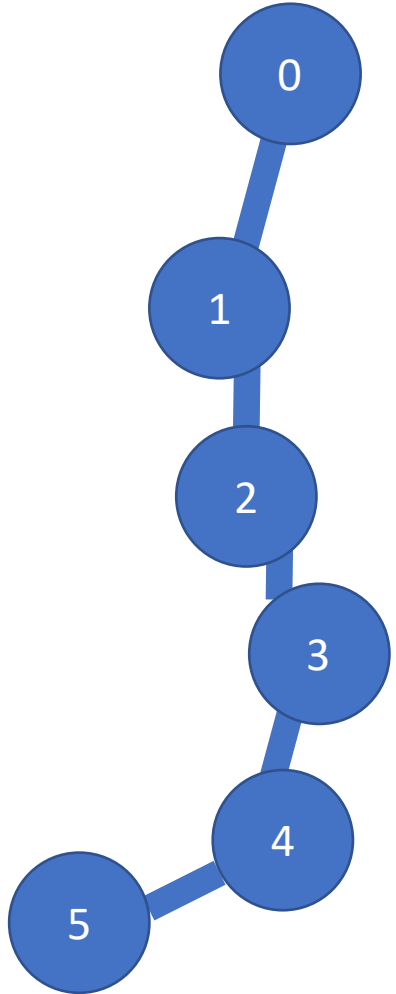
- Weighting keeps trees shallow!
- But we can still do *find(5)* a thousand times on that same tall-ish tree, and we re-climb the same path every single time.
- Since we're already walking that path to answer the query, why not just make every node on the path point directly to the root? That way we never have to repeat our work

Path compression: the idea

- While we're doing *find(p)*, we walk from p up to the root anyway – so grab every node we pass on the way.
- Once we know the root, point every one of those nodes **directly at the root**, instead of at whatever they used to point to.
- The next *find()* on any of those nodes is now one hop, not a whole climb.

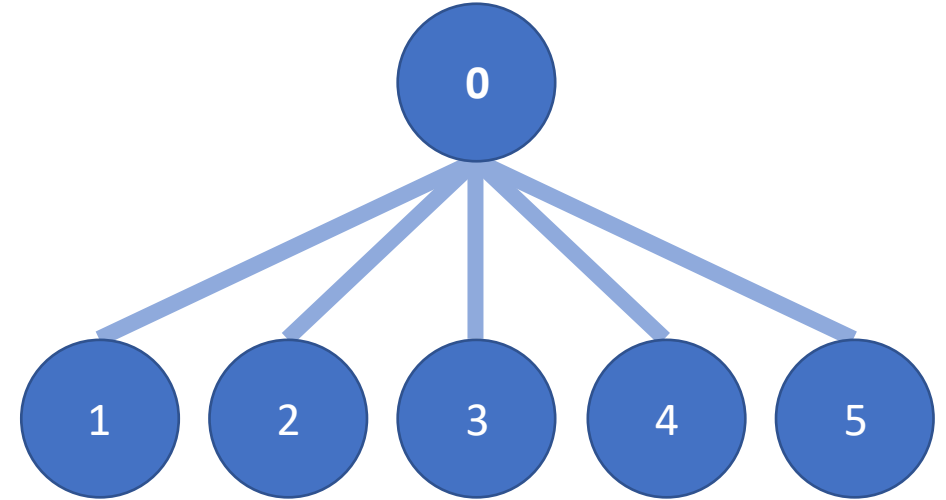
Flattening the tall tree

Before: find(5)



climbs 5 hops, every single time

After: path compression



every node now points straight at 0 — find is 1 hop

This is about as good as you can do

- Weighted quick-union alone: $O(\log n)$ per operation.
- Weighted quick-union + **path compression**: $\alpha(n)$ per operation, where α is the inverse Ackermann function.
- What's the inverse Ackermann function? We'll talk about it *briefly* later in the semester, but the key thing is that it grows *very very very very* slowly.
- $\alpha(n) < 5$ for any n you will ever encounter in practice
- This is the standard, near-optimal union-find implementation used in practice.